

Ch-1 Introduction

Learning from data

What is data?

Why do we learn from data?

- understand, analyse & act/predict
- extract patterns, semantics & generally meta-data from raw signals.

How?

→ Statistical learning. [ML + statistics]

- Deep learning belongs to ML approaches

→ Analytical solutions might not exist

→ Expensive closed-form solution.

Deep learning is a family of methods to learn data representations, inference and sampling models.

Possibilities: features, embeddings, classifiers, regressors, predictors & samplers.

Ex: Given RGB image detect lanes on road.

Algorithmic solution

Off-line phase

1. build feature based lane model
2. Restrict ROI based on scene geometry

On-line phase

1. capture img & crop it according to ROI
2. Run sliding window to measure diff b/w reference & extracted histograms.
3. Normalise result to give it a probabilistic interpretation & accept all pixels below some dist. threshold as lane.

(Learning based) Algorithmic sol.

1. Offline phase [training]
collect & annotate images with lanes.
2. Extract HOG features to represent lanes & background.
3. Train binary classifier [Ex: linear SVM or Random Forest]

Online phase [inference]

1. Capture image & run a sliding window to classify each pixel as lane or background.

Algorithmic solution [Deep Learning]

Offline phase

1. collect & annotate images with lanes
2. Train a deep neural net. to detect lanes on image.

Online phase

1. Capture image & pass it to Network to classify each pixel as lane/background.

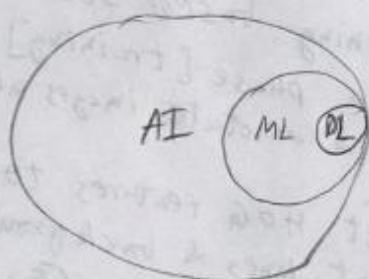
Applications of deep Learning:

- obj. detection
- counting
- satellite image segmentation
- speech recognition
- point cloud labelling.

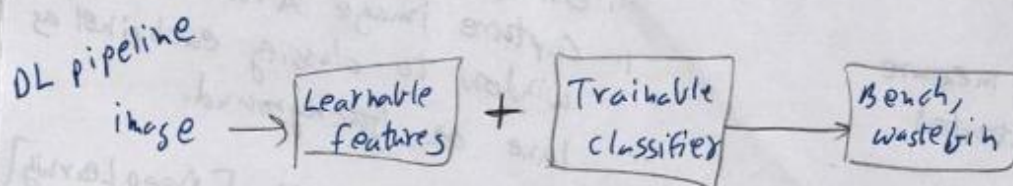
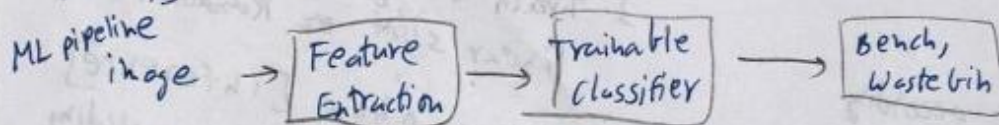
- text generation
- Audio separation
- text spotting

DL definition:

- Parametric model with non-linear functions. [As data distr. are usually highly non-linear]
- in practice it is a composition of linear & non-linear functions
- Goal: Learn hierarchical representations [Ex: edges, shapes, parts etc]
- It has 1 or more objectives to learn parameters from data.
- Represented by deep neural networks
 - convolution
 - stochastic gradient descent & chain rule $\rightarrow$ differentiable



DL V/S ML



- representation learning & training of classifier takes place at same time. [End-End Learning]
- There have been 2 AI winters in the past

McCulloch-Pitts Neuron [Threshold Logic Unit]

i/p $x_i \in \{0, 1\}$; binary o/p $y \in \{0, 1\}$

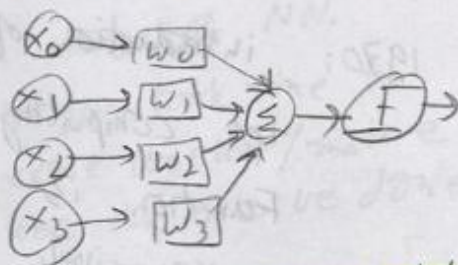
Excitatory / inhibitory i/p represented by weights/params

$$w_i \in \{-1, 1\}$$

Mapping is represented by a threshold function $f: \mathbb{R}^D \rightarrow \mathbb{R}$

where

$$f(x) = \begin{cases} 0 & \text{if } w \cdot x < t \\ 1 & \text{otherwise} \end{cases}$$



Observations

- weighted sum as o/p
- No learning algo, weights & thresholds (T) predefined [computed by hand]

Limitations

- 0/1 i/p
- hand defined parameters
- cannot implement XOR / complex gates.

Motivation

Dendrites: inputs

Soma: sum of inputs

Axon: o/p sig. propagation to other neurons.

Synapse: connection point to other neurons.

Perceptron

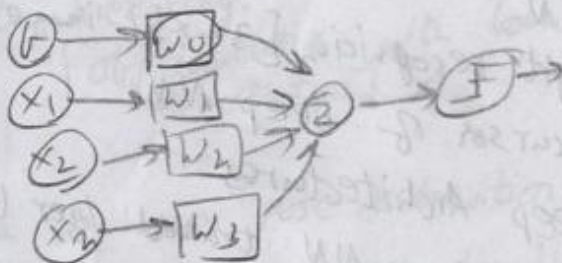
- Linear classifier for binary problems
- real valued i/p vector
- supervised learning on n i/p, o/p pairs

$$D = \{(x_1, y_1) \dots (x_n, y_n)\}$$

Represented by threshold function $f: \mathbb{R}^D \rightarrow \mathbb{R}$

$$f(x) = \begin{cases} 1 & \text{if } w \cdot x + b \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

- Learning rule: parameter w update is proportional to i/p & diff. b/w prediction $f(x)$ & ground-truth y_i



Algorithm

1. initialise parameters w , set learning rate η & threshold.
2. update weights: $w_j(t+1) = w_j(t) + \eta (y - f(x)) x$
3. Iterate the update until convergence.

What are diff. b/w Perceptron & Threshold logic unit?

Limitations

- Single layer perceptron cannot implement XOR function
- Multi layer perceptron capable of approximating XOR

Back propagation

1970: introduction of reverse mode of automatic differentiation for computing derivatives of a function composition. Function is represented as a graph & chain rule is recursively applied to get derivatives.

1974: Definition of process of training an artificial ~~neural~~ NN through back-propagation of errors.

Neocognitron

- hierarchical multilayered ANN
- partial shift invariance.
- Precursor of CNN.

LeNet

- CNN
- Digit recognition [32x32 image i/p]
- Precursor of Deep NN.

Deep Architectures

- 2006: Deep NN trained layer by layer
- 2007: Deep NN architecture proposed.

SVM
MRF
Manifold learning &
other approaches were
drivers of research in
ML.

Alex Net

- brought Deep Learning revolution
- Max pooling → ReLU → data augmentation SGD with momentum.

Recent transformer architectures are deep & complex.
reaching about 0.5B parameters.

Ingredients that got DL success

1. Computational power & infrastructure
2. Available amount of data
3. Algorithms & community.

[Matrix-matrix
Matrix-vector multipl
require GPUs]

2 ways to collect data

- Web scraping
- crowd sourcing [data could have bias]

- lack of data & pre-processed data inhibits usage of NN.

- Algo. took time to come-together/mature for task to be done.

Compute power

Motivation: to build chips equal to human brain
MIPS [Millions of Instructions per second]
FLOPS [Floating Point operations per second]

Data

MNIST: Handwritten digits
28x28 grayscale img.

CIFAR: Canadian Institute for Advanced Research
10,000 32x32 RGB img.

ImageNet: based on WordNet hierarchy.
256x256 images [obj localisation with bounding box annotation]

PASCAL VOC

COCO: obj. detection, segmentation & human pose estimation.

Youtube bounding boxes: Youtube videos with single obj bounding box

KITTI: ^{large-scale} Driving video database [2 video cameras]

Reading comprehension: question answering dataset.
look question-ans. pairs

Natural language inference: Natural Lang. inference corpus

Machine translation: translating text from one lang. to other lang.
570K human written English sentence pairs.

NN doesn't work well for audio as amount of processing required is large. 40 KHz / 40k samples/s need to be processed.

Comprehensive Question

1. What is McCulloch-Pitts Neuron?
2. Why was 1st perceptron criticised?
3. What was the 1st step toward DL revolution?

1. What are 3 important factors that made Deepneural n/w well-function?
2. Based on AI statistics, are we going to face new AI winter?

Ch2 ML- Algorithms

A program is said to learn from Experience E WRT some class of tasks in T , ~~as measured by~~ ^{and perf measure} P , if its performance at tasks in T , as measured by P , improves with experience E .

Tasks T

1) Classification (Ex: image/face recognition)

- A func. $f: \mathbb{R}^n \rightarrow 1, \dots, K$ that maps a sample represented by a set of n -dim features, to a categorical o/p.

- $y = f(x)$; is the category prediction for i/p x .

- o/p can be described on diff. ways ex: probabilistic.

How do we define mapping f for probabilistic o/p?

2) Regression

- A func. $f: \mathbb{R}^n \rightarrow \mathbb{R}$ that predicts a numerical value.

What about high dim. o/p? Def. mapping function.

Ex: depth estimation, audio gen., predict no. of postcomments

Etc. - Machine translation (De to Eng), structured prediction, denoising

Performance measure P

Refers to evaluation of ML algorithms.

- Requirement: a quantitative measure of ability to perform a task.

- Same metrics across diff. ML algos helps compare algorithms.

Task related measures:

- Classification: Accuracy, precision, recall, sensitivity, specificity, F1 score

- Regression: mean absolute error MAE, MSE, RMSE, L_2 -norm

- For some tasks not clear what to measure (Ex: image/audio generation)

Experience E

Due to complexity, it is currently unlikely to experience the whole world using a computer program. For that reason ML algos experience datasets.

A dataset is a collection of samples. It is divided into training and test sets. Assumption is samples are independent. A test set are i.i.d. Additionally, all data are drawn from same prob. distribution.

Experiencing a dataset is connected to type of learning.

Learning categories

1) Supervised learning $\rightarrow D = \{(x_1, y_1), \dots, (x_n, y_n)\}$, where we have to access pairs of i/p features $x_i \in \mathbb{R}^d$ & target y_i .

- i/p features can be image pixels while target/label y can be

class category, scalar, vector & tensor o/p.

- i/p data space X ; $x_i \in X$ & o/p target space Y ; $y_i \in Y$ varies

WRT application

- training is same, i.e. predict y from x as $P(y|x)$

What is disadv. of supervised learning?

- time consuming for labelling [human errors]
- definition of labels & intervention needed to standardise.

2) Unsupervised learning: learn data prob. distribution $P(X)$ in implicit/explicit way.

Algo has to understand data without guidance.

3) Weakly-supervised learning $\rightarrow$ labels of training set $y_i \in Y$, not available due to annotation complexity. Instead training set has "cheaper"

labels $y_i \in \hat{Y}$ for each i/p $x_i \in X$.

Ex: image based obj detection, where training sample only has obj category annotation, not location within image.

4) Semi-supervised learning: $D_1 = \{(x_1, y_1), \dots, (x_k, y_k)\}$; $k \ll n$
[labeled + unlabelled data] $D_2 = \{x_1, \dots, x_n\}$

Ex: Digit classification with limited data.

5) Self-supervised learning $\rightarrow D = \{x_1, \dots, x_n\}$

Data acts as supervision. Proxy tasks are used for representation learning.

6) Reinforcement learning:

More than experiencing fixed data set, interaction with environment creates experience.

Linear Regression

Goal: Predict the scalar $y \in \mathbb{R}$

Design prediction as linear function $f(\cdot)$ of i/p:

$$\hat{y} = f(x; w) = w^T x \quad ; \quad w \in \mathbb{R}^n \quad w \rightarrow \text{parameters vector}$$

i/p: feature vector $x \in \mathbb{R}^n$

Learning = finding parameter values (w) that results in correct prediction $\hat{y}$, i.e. $\hat{y} = y$ for i/p vector x

Linear function $f(\cdot)$ can be formulated with additional bias b .

$$\Rightarrow \hat{y} = w^T x + b \quad ; \quad \text{bias } b \rightarrow \text{predefined}$$

The mapping from i/p to prediction here becomes

an affine function.

split into $\left\{ \begin{array}{l} \text{training set: } x^{(\text{train})} \in \mathbb{R}^{n \times n}, y^{(\text{train})} \in \mathbb{R}^m \text{ for } m \text{ samples} \\ \text{test set: } x^{(\text{test})} \in \mathbb{R}^{n \times n}, y^{(\text{test})} \in \mathbb{R}^m \end{array} \right.$

: MSE b/w prediction ($\hat{y}$) & ground truth (y)

$$MSE = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2$$

is to regress the value $\hat{y}$ from i/p x vector.

Learning algo is allowed to experience only training-set to improve parameters w .

- Perf. measure is same for training & testing, i.e. MSE.
goal: minimise MSE on training set that results in low MSE on test set.

How to find min? Take the derivative of MSE set it to 0

[Parameters = weights (other than bias) in neural networks
error function = objective/loss function in neural networks]

Performance measure (loss function): measures how good our prediction is.

Optimization: is the process of changing the model parameters until selecting the best one.

What are the values that correspond to best parameters?
values that make gradient / derivatives = 0.

Derivation:

$$MSE = \frac{1}{m} \sum_{i=1}^m \left(\hat{y}_i^{(train)} - y_i^{(train)} \right)^2 = \frac{1}{m} \sum_{i=1}^m \left(W^T x^{(train)} - y_i^{(train)} \right)^2$$

in matrix notation [remove $1/m$ as it isn't significant for minimizing]

$$MSE = \left(X^{(train)} W - y^{(train)} \right)^T \left(X^{(train)} W - y^{(train)} \right)$$

Expanding this further [take inside the Transpose & multiply]

$$MSE = \left(W^T X^{(train)T} - y^{(train)T} \right) \left(X^{(train)} W - y^{(train)} \right)$$

$$= W^T X^{(train)T} X^{(train)} W - W^T X^{(train)T} y^{(train)} - y^{(train)T} X^{(train)} W + y^{(train)T} y^{(train)}$$

$$= W^T X^{(train)T} X^{(train)} W - 2 W^T X^{(train)T} y^{(train)} + y^{(train)T} y^{(train)}$$

$$(\because y^{(train)T} X^{(train)} W = (y^{(train)T} X^{(train)} W)^T = W^T X^{(train)T} y^{(train)})$$

to find min. of MSE take gradient & set to 0.

$$\nabla_W MSE = 0 \Rightarrow$$

$$\nabla_W (W^T X^{(train)T} X^{(train)} W - 2 W^T X^{(train)T} y^{(train)} + y^{(train)T} y^{(train)}) = 0$$

$$\Rightarrow 2 X^{(train)T} X^{(train)} W - 2 X^{(train)T} y^{(train)} = 0$$

$$\Rightarrow X^{(train)T} X^{(train)} W = X^{(train)T} y^{(train)}$$

$$\Rightarrow W = \underbrace{\left(X^{(train)T} X^{(train)} \right)^{-1}}_{\text{pseudo inverse}} X^{(train)T} y^{(train)}$$

Note: In DL it is common to have approximations of the sol.
The gradient though is still necessary.

What happens if we increase no. of samples significantly?

Generalization is the ability to perform well on unobserved samples (test set)

Observed samples = training + validation set

Performance reflected in training & test error. [generalisation error]
reducing this error is the main challenge in ML

The training & testing data is sampled from same distribution.

↓
used to
learn parameters
of ML algo

↓
Algo then evaluated
on test set

goal: min. training error & close
gap b/w train & test
errors.

In data gen. process we always make i.i.d assumption, about train & test.

↓
independent identically distributed

Based on min. training & test error, we deal with

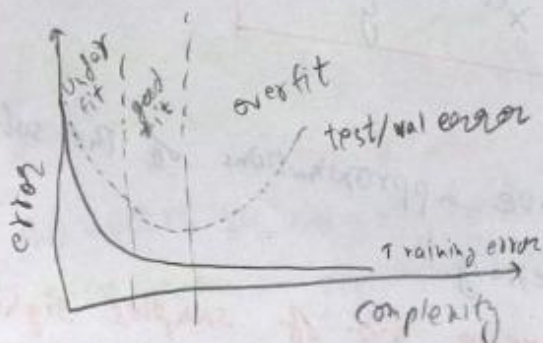
- overfitting: training error is small & test error is large.
can be due to too many params.
- underfitting: training error cannot decrease
can be due to too few model parameters
- capacity: determines how complex mappings the model can learn.
can help avoid under-fitting & prevent over-fitting.
In neural nets it can be described by n.o. of n/w params.

How is model capacity connected to under/over-fitting? graphically illustrated

effective capacity $\leq$ representational capacity
(in practice) (theoretical)

- Finding right capacity is one way of preventing under/over fitting

For DL algos actual capacity measurement is ltd by optimisation, for which there is no ^{sufficient} theoretical understanding.



Regularisation

It is a constraint on the objective/loss func. that indicates a preference on the solution.

Ex: weight decay. (L2 norm of the params)
adds weight on fewer features / shall slope on regression example.

linear regression with L2 regularizer

$$(w x^{(train)} - y^{(train)})^T (w x^{(train)} - y^{(train)}) + \lambda w^T w$$

influence of regulariser

- reduces ~~training~~ ^{test} error & not ~~test~~ ^{training} error.

Hyperparameters

- is a param. whose value is set prior to learning.
- can stay fixed / change based on pre-defined rule.
- affect model's behaviour
- finetuning hyperparameters is tedious & requires extra computational power.
- Automate machine learning (AutoML) provides a solution to automatically choose hyperparameters.

Ex: λ in linear regression with L2 regularizer influences weight decay.

- cannot be chosen on training set as it will cause overfitting.
- instead create validation set from training set.
split rule is usually 80% for training & 20% for validation
- validation error $\approx$ test error
- use cross validation for smaller data-sets. to split data into training & validation, for obtaining reliable error estimate.
k-fold cross validation: splits data into k non-overlapping segments. At each round 1 segment is training set & other k-1 segments are test-set. test error = average across k trials.
- consumes extra computational costs.

Maximum Likelihood Estimator

Likelihood measures how likely/plausible the value of a parameter of a statistical model is, given observed data.

Not same as probability but can be formulated as prob. using Bayes theorem.

Estimator is a function of the data, i.e. $\hat{w} = g(x^1, \dots, x^m)$ which outputs estimate of param that generated it.

Good estimator: $\hat{w} = w$ [estimates as close as to true parameters]

function estimators / point estimators:

↳ in function space as function is defined by set of parameters

Max: we look for best fit in terms of values for model params. i.e. maximise the likelihood function.

Definition: m data samples $D = \{x_1, \dots, x_m\}$ or $D = \{(x_1, y_1), \dots, (x_m, y_m)\}$

drawn from unknown distr. $p_{data} = (x)$.

let $p_{model} = (x; w)$, be a parametric model / family of distr. that describe the data.

maps i/p x to a real n.o. that estimates true data prob. $p_{data}(x)$

in practice $\hat{p}_{data} = (x)$ as we have access to only empirical data distribution. [training data]

Although we assume we know the prob. distr. func. we do not know parameters of it.

∴ we have to estimate w that best explains the data.

$$w_{ML} = \underset{w}{\operatorname{argmax}} p(D; w) = \underset{w}{\operatorname{argmax}} \prod_{i=1}^m p_{model}(x_i; w)$$

↳ product
here cause instability

due to instability max. the log of w_{ML} , i.e.

$$w_{ML} = \underset{w}{\operatorname{argmax}} \sum_{i=1}^m \log p_{model}(x_i; w)$$

Now dividing by n.o. of samples gives us expectation WRT empirical data prob. distr. $\hat{P}_{data}$ (training set), i.e.

$$W_{ML} = \underset{w}{\operatorname{argmax}} \frac{1}{n} \sum_{i=1}^n \log P_{model}(x_i; w)$$

$$= \underset{w}{\operatorname{argmax}} E_{x \sim \hat{P}_{data}} \log P_{model}(x; w)$$

Interpretation:

Minimize dissimilarity b/w $\hat{P}_{data}$ and model distr P_{model} .
i.e. to learn model parameters that best explain the training data.

Dissimilarity is measured using KL divergence
KL-divergence measures difference of true prob. distr P from other reference distr. Q .

$$D_{KL}(P \parallel Q) = - \sum_{x \in \mathcal{X}} P(x) \log \left(\frac{Q(x)}{P(x)} \right) = \sum_{x \in \mathcal{X}} P(x) \log \left(\frac{P(x)}{Q(x)} \right)$$

what is it for cont. distr?
what are true & ref. distr in our prob?

$$D_{KL}(\hat{P}_{data} \parallel P_{model}) = \sum_{x \in \mathcal{X}} \hat{P}_{data}(x) \log \left(\frac{\hat{P}_{data}(x)}{P_{model}(x; w)} \right) \rightarrow \text{interpreted as info gain if we use } P_{model} \text{ instead of } \hat{P}_{data}$$

This is the expectation of log diff b/w $\hat{P}_{data}(x)$ & P_{model}

$$\Rightarrow D_{KL}(\hat{P}_{data} \parallel P_{model}) = E_{x \sim \hat{P}_{data}} \left[\log \left(\frac{\hat{P}_{data}(x)}{P_{model}(x; w)} \right) \right]$$

$$= E_{x \sim \hat{P}_{data}} [\log \hat{P}_{data}(x) - \log P_{model}(x; w)]$$

objective: estimate w that minimises KL divergence.

omit $\log \hat{P}_{data}(x)$ in minimisation as it is independent of w .

$$\Rightarrow W_{KL} = \underset{w}{\operatorname{argmin}} E_{x \sim \hat{P}_{data}} [\log P_{model}(x; w)]$$

→ minimising KL divergence = maximising MLE for large n.o. of samples.

→ often we will minimise -ve log-likelihood (NLL) instead of maximising log-likelihood.

→ minimising KL divergence = minimising cross entropy (CE) b/w 2 distr.

MLE conditional:

In supervised learning, we have cond. prob. $p(y|x;w)$, where aim is to predict y from x based on W (model params)

can use MLE here as well

$$w_{ML} = \arg \max_w p_{\text{model}}(y|x;w)$$

$$= \arg \max_w \frac{1}{n} \sum_{i=1}^n \log p_{\text{model}}(y_i|x_i;w)$$

$$= \arg \max_w E_{x,y \sim \hat{p}_{\text{data}}} \log p_{\text{model}}(x;w)$$

Linear Regression as MLE:

prediction is a linear function $f(\cdot)$ of i/p.

$$\hat{y} = f(x;w) = w^T x$$

New formulation:

- instead of single valued pred. funct. we build cond. distr. func.

$$p_{\text{model}}(y|x)$$

- assume same i/p x can be mapped to diff. y , which are closer to each other.

Then question is how probable estimate $\hat{y}$ is?

- assume $\hat{y}$ comes from normal distr. with mean μ & var. σ^2

$$\text{cond. distr. } p_{\text{model}}(y|x;w) = \mathcal{N}(\hat{y}; y, \sigma^2)$$

log-likelihood is then

$$\sum_{i=1}^n \log p_{\text{model}}(y_i|x_i;w) = \sum_{i=1}^n \log \left(\frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(\hat{y}_i - y_i)^2}{2\sigma^2}} \right)$$

$$= \sum_{i=1}^n \log \left(\frac{1}{\sqrt{2\pi}\sigma} \right) + \sum_{i=1}^n \log e^{-\frac{(\hat{y}_i - y_i)^2}{2\sigma^2}}$$

$$= \sum_{i=1}^n \log (2\pi\sigma^2)^{-\frac{1}{2}} - \sum_{i=1}^n \frac{(\hat{y}_i - y_i)^2}{2\sigma^2} = -\frac{n}{2} \log(2\pi\sigma^2) - \sum_{i=1}^n \frac{(\hat{y}_i - y_i)^2}{2\sigma^2}$$

$$= -\frac{n}{2} \log(2\pi) - n \log(\sigma) - \sum_{i=1}^n \frac{(\hat{y}_i - y_i)^2}{2\sigma^2} \quad ; \text{ now subst. } \hat{y} = w^T x_i$$

$$= -\frac{n}{2} \log(2\pi) - n \log(\sigma) - \sum_{i=1}^n \frac{(w^T x_i - y_i)^2}{2\sigma^2} = -\frac{n}{2} \log(2\pi) - n \log(\sigma) - \frac{(Xw - y)^T (Xw - y)}{2\sigma^2}$$

⊕ calc gradient of ⊕ we get,

$$\nabla_w \left[-\frac{n}{2} \log(2\pi) - n \log(\sigma) - \frac{(xw - y)^T (xw - y)}{2\sigma^2} \right] = 0$$

$$\nabla_w [(xw - y)^T (xw - y)] = 0$$

$$\nabla_w [w^T x^T x w - y^T x w - w^T x^T y + y^T y] = 0$$

$$\nabla_w [w^T x^T x w - 2w^T x^T y + y^T y] = 0$$

how? $\rightarrow 2x^T x w - 2x^T y = 0$

$$x^T x w = x^T y$$

$$w = (x^T x)^{-1} x^T y$$

$\rightarrow$ i.e. MSE can be interpreted as max. likelihood estimation.

rough work

$$\begin{aligned} & (xw - y)^T (xw - y) \\ &= x^T x w^T x w - 2x^T y^T w + y^T y \\ &= 0 \end{aligned}$$

Optimisation

ML ADL algo depend on a loss function [objective]

Optimisation algorithm minimizes loss / maximises some other objective.

Gradient based optimisation is most common way of looking for min/max of a function.

We find values of a parameter using grad descent to minimise loss.

Gradient descent: is an iterative optimisation algorithm to find a function's minimum. Function can be regression/classification.

$L(x, y, w) \rightarrow$ loss function parametrised by w
grad. descent iteratively updates parameters(w) in opposite direction of gradient $\nabla_w L(x, y, w)$.

$\eta \rightarrow$ learning rate [influences how much an update modifies the parameters]

goal: to reach global min. of loss func. or local min. if loss func. is non-convex.

How to compute gradients?

1. differentiate
2. Finite difference (numerical approach): Approximation.

1) Batch gradient descent [std (vanilla) grad. descent]

Gradient of loss $L(x, y, w)$ WRT parameters w is computed on whole training set.

$$w = w - \eta \nabla_w L(x, y, w)$$

- when dataset is large update can be slow
- cannot perform update if dataset doesn't fit into memory.
- converges to global min. after sufficient iterations.
- converges to local min. if loss is non-convex.
- convergence is slow as it considers gradients of all points which might point in different directions.

2) Stochastic gradient descent

Parameter update is performed for each training sample separately.

$$w = w - \eta \nabla_w L(x_i, y_i, w)$$

- fast update with fast variance.
- can be used for online training
- limitation is due to high fluctuations, caused due to lack of gradient averaging during update step.
- can reach global/local min. when lowering/adjusting learning rate & iterating longer than batch gradient descent.

best compromise b/w efficiency & stable training

3) Mini-batch gradient descent

Performs gradient update for every mini-batch of n samples

$$w = w - \eta \nabla_w L(x_{(i:i+n)}, y_{(i:i+n)}, w)$$

- smaller variance of param. updates compared to SGD & hence faster / more stable convergence.
- mini-batch gradients are computed efficiently, hence faster than SGD.

SGD & mini-batch are used interchangeably but refer to mini-batch version mostly.

Challenges of grad. descent

- fixed learning rate: low LR results in slow convergence
large LR results in fluctuations around minimum
A lack of convergence.
- Adaptive LR: LR can be decreased based on some rule.
but not straight forward which technique is right.
- saddle points: point where slope in orthogonal directions are all close to zero, but is a local min.
grad. descent can get stuck at these points.
- multiple LR: all params don't update in same way, but LR is same for all.
Using diff. LR per param can improve convergence but it increases problem complexity.

Classification

Motivation: is to differentiate b/w finite set of categories [classes].

Logistic regression transforms prob. from regression to classification, with support of a function with bounded o/p.

Convert continuous o/p of a bounded function to classification output [category prediction]

Logistic regression [2 classes]

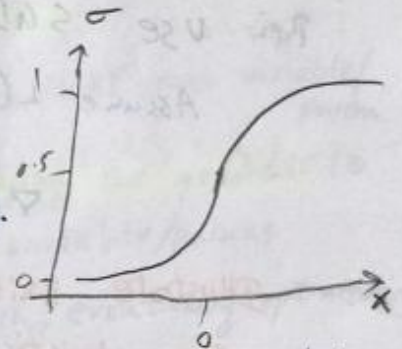
$y \in \{0, 1\} \rightarrow$ for 2 class prob

$\hookrightarrow$ o/p

map i/p x to one of these 2 values (0/1).

logistic func. does this. $[\sigma(\cdot)]$

threshold at 0.5 is well suited to decide b/w 2 classes.



$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$\rightarrow$ eqn for logistic function.

$$\frac{\partial \sigma}{\partial z} = \sigma(z)(1 - \sigma(z)) ; z \rightarrow \text{scalar}$$

parametrising with w

$$h_w(x) = \frac{1}{1 + e^{-wx}}$$

$w \rightarrow$ regression parameters

$x \rightarrow$ i/p vector

$\hookrightarrow$ o/p of $h_w(\cdot)$ is probability of being class 1

probabilistic interpretation of $h_w(\cdot)$ is now

$$h_w(x) = p(y=1|x;w) = 1 - p(y=0|x;w)$$

We can now rely on set of n training pairs & minimise MSE like in linear regression

$$MSE = \frac{1}{n} \sum_{i=1}^n (h_w(x_i) - y_i)^2$$

→ no analytical sol exists. hence use MLE on training set.

minimisation of MSE:

It is a non-convex problem due to the sigmoid.

goal: maximise ~~$h_w(x)$~~ $p(y=1|x;w)$

conditional MLE: $w^* = \arg\max_w \frac{1}{n} \sum_{i=1}^n \log p(y_i|x_i;w)$

$$w^* = \arg\max_w \frac{1}{n} \sum_{i=1}^n y_i \log(h_w(x_i)) + (1-y_i) \log(1-h_w(x_i))$$

We can instead minimise NLL (-ve log likelihood)

$$w^* = \arg\min_w - \frac{1}{n} \sum_{i=1}^n y_i \log(h_w(x_i)) + (1-y_i) \log(1-h_w(x_i))$$

Then use SGD

Assume $L(x, y, w) = \frac{1}{n} \sum_{i=1}^n -y_i \log(h_w(x_i)) - (1-y_i) \log(1-h_w(x_i))$

$$\nabla_w L(x, y, w) = \sum_{i=1}^n (y_i - h_w(x_i)) x_i$$

Illustrate limitation of Linear Regression to solve 2-class classification?
Draw logistic regression to show how it separates data into 2 categories.

Other classification problems: k-nearest neighbors, naive Bayes classifier, SVM, Decision tree

Learning algo requires

- 1) Dataset
- 2) Objective func

- 3) optimisation
- 4) model

Ch-3 Feed Forward Networks & Back prop.

Feed fwd NN is a composition of operations that can be represented by nodes of a directed graph. The NN approximates the mapping function f which consists of learnable parameter set.

$$f: X \rightarrow Y$$

Ex: $X \rightarrow \text{image}$

$Y \rightarrow \text{output space} = \text{object category}$

Learning the mapping requires training data like ML algos. give i/p $x \in X$, network predicts $y = f(x; w)$

composition of functions: composition of differentiable functions.

Ex: 4 layer network: $f_1 = \sigma(w_1 x)$ nonlin. activation
 $f_2 = \text{pool}(f_1)$ ~~max~~ pooling / subsamples
 $f_3 = \sigma(w_2 f_2)$ nonlin. activation
 $f_4 = \text{softmax}(w_3 f_3) \rightarrow \text{class prediction}$

$\rightarrow$ operations inside NN ~~have to~~ ^{can} be linear/non-linear but have to be differentiable.

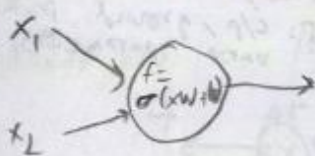
Learning parameters of NN / training a NN refers to minimising loss function L . The optimisation $\nabla_w L$ involves:

- ① Backpropagation: compute gradient of loss WRT each variable/param.
- ② Gradient descent: rule of iteratively applying the gradients to update value of the variable/params.

Learning can also be done using: random search, evolutionary optimisation, bayesian optimisation.

NN can be visually represented artificial neurons / layers of them.

Artificial Neuron: is a composition of operations.



1. an i/p multiplied with weights
2. bias included
3. nonlinear activation ($\sigma \rightarrow \text{sigmoid}$) takes in the sum

Neurons like these in many layers communicate with each other through their activations.

Multilayer Perceptron (MLP)

Std. feed forward network is an MLP.

Ex: Point Net

- It is a shallow network [i/p $\rightarrow$ hidden $\rightarrow$ o/p layers]

Why was it challenging to add hundreds of hidden layers?

- challenging due to n.o. of calculations & also due to non-linear activation functions.

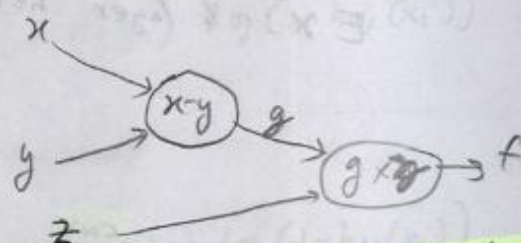
NN as a computational graph

Computational graph is a directed graph

- drawing a graph is useful when computing its weights.

nodes = operations
edges = data flow

Ex: $f(x, y, z) = (x - y)z$



We represent NN as computational graphs with forward & backward pass.

Forward pass: evaluating a function/inter a set of inputs.

It is process of making a prediction, using NN. graph variables learn using grad. based optimisation

Backward pass: calculates gradients of all variables.

It is done after the forward pass.

Given a NN which terms are updated / not updated?

Ex: in above graph x, y are i/p, g is intermediate term & f is output. z is a model term that is updated / learned.

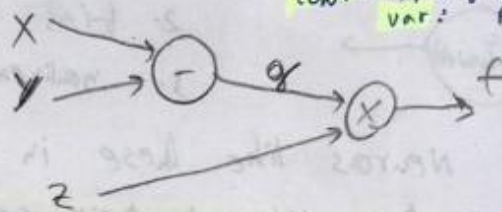
Grad computation is automatised by expressing NN as a computational graph.

Back prop example:

$$f(x, y, z) = (x - y)z$$

for $x=5, y=2, z=-4$

$$g = x - y$$
$$f = g \times z$$



FWD pass
 $x=5, y=2, z=-4 \Rightarrow f=-12$
which are const & variables?
const: i/p, ground truth
var: parameters.

Derivatives of the graph tell us how input influences the o/p.

calculated using partial derivatives of o/p WRT i/p.

i.e. $\frac{\partial f}{\partial x}$, $\frac{\partial f}{\partial y}$, $\frac{\partial f}{\partial z}$ using the chain rule from end to start.

$\frac{\partial f}{\partial f} = 1$

$\frac{\partial f}{\partial z} = z$, $\frac{\partial f}{\partial z} = g$ ($\because f = g \times z$ & $g = x - y = 3$)

$\frac{\partial g}{\partial x} = 1$, $\frac{\partial g}{\partial y} = -1$ ($\because g = x - y$)

finally:

$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial g} \frac{\partial g}{\partial x} = z \cdot 1 = -4$; $\frac{\partial f}{\partial y} = \frac{\partial f}{\partial g} \frac{\partial g}{\partial y} = z \cdot -1 = -z = 4$

$\frac{\partial f}{\partial z} = g = x - y = 3$

here $\frac{\partial f}{\partial g}$ = upstream gradient & $\frac{\partial g}{\partial x}$ = local gradient

similar for $\frac{\partial f}{\partial z} = \frac{\partial f}{\partial f} \frac{\partial f}{\partial z}$ & $\dots \frac{\partial f}{\partial y} = \dots$

$\frac{\partial f}{\partial g} = \frac{\partial f}{\partial f} \frac{\partial f}{\partial g}$

local gradient = derivative of o/p WRT i/p of a node

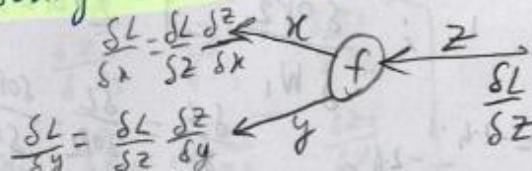
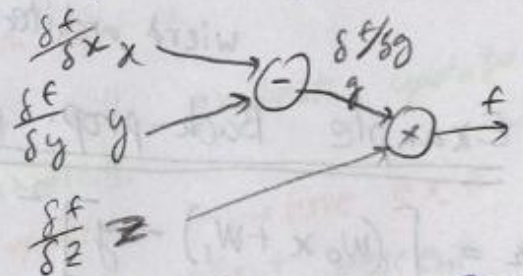
upstream gradient = derivative of loss function WRT o/p of current layer.

forward pass = operation takes i/p to produce o/p.

backward pass = local gradients are computed

At beginning of backward pass upstream gradient is derivative of loss function, WRT o/p of last graph operation.

For a neural network loss is usually the last operation. It comes after prediction.



we back-propagate to calculate gradients of the network parameters

How do we use gradients to update network params?

→ gradient descent.

↳ repetitive process to avoid computational overhead.

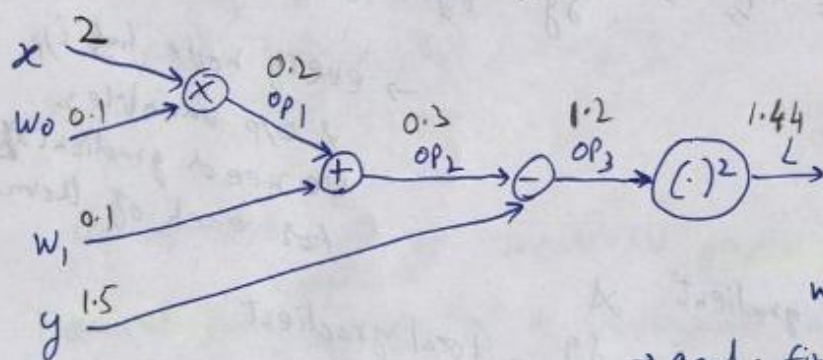
Does back-prop. apply to unsupervised learning?

Yes

Back prop is ^{also} applicable with regulariser / w/o regulariser
wierd architectures; with / w/o labels ... etc.

Example Back-prop. for linear regression.

$$\text{error} = \left[\underbrace{(w_0 x + w_1)}_{\text{Prediction}} - \underbrace{y}_{\text{ground truth}} \right]^2 \quad \left[\text{mean ignored as it doesn't affect pos. of minima} \right]$$



$$OP_1 = w_0 x ; OP_2 = OP_1 + w_1$$

$$OP_3 = y - OP_2 ; L = OP_3^2$$

initial values

$$w_0 = w_1 = 0.1 ; x=2 \times y=1.5$$

→ goal: find best parameters. & therefore

$$\frac{\partial L}{\partial w_0}, \frac{\partial L}{\partial w_1}$$

$$\text{Node } (.)^2 \rightarrow \frac{\partial L}{\partial L} = 1 \quad (\text{upstream})$$

$$\frac{\partial L}{\partial OP_3} = 2 \cdot OP_3 = 2 \cdot 1.2 = 2.4 = \frac{\partial L}{\partial L} \frac{\partial L}{\partial OP_3}$$

$$\text{Node } (-) \rightarrow OP_3 = y - OP_2$$

$$\frac{\partial OP_3}{\partial OP_2} = -1 ; \frac{\partial OP_3}{\partial y} = 1 \quad [\text{local gradients}] ; \text{upstream grad here} = \frac{\partial L}{\partial OP_3}$$

$$\frac{\partial L}{\partial OP_2} = \frac{\partial L}{\partial OP_3} \frac{\partial OP_3}{\partial OP_2} = 2.4 \cdot -1 = -2.4 ; \frac{\partial L}{\partial y} = \frac{\partial L}{\partial OP_3} \frac{\partial OP_3}{\partial y} = 2.4 \cdot 1 = 2.4$$

$$\text{Node } (+) \rightarrow \frac{\partial OP_2}{\partial OP_1} = 1 ; \frac{\partial OP_2}{\partial w_1} = 1 \quad [\text{local gradients}] ; \text{upstream grad} = \frac{\partial L}{\partial OP_2} = -2.4$$

$$\frac{\partial L}{\partial OP_1} = \frac{\partial L}{\partial OP_2} \frac{\partial OP_2}{\partial OP_1} = -2.4 \cdot 1 = -2.4$$

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial OP_2} \frac{\partial OP_2}{\partial w_1} = -2.4 \cdot 1 = -2.4$$

Node (x) $\rightarrow$

$$\frac{\partial \text{op}_1}{\partial x} = w_0 = 0.1 ; \frac{\partial \text{op}_1}{\partial w_0} = x = 2 \quad [\text{local grad}] \quad \text{upstream} \rightarrow \frac{\partial L}{\partial \text{op}_1} = -2.4$$

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial \text{op}_1} \cdot \frac{\partial \text{op}_1}{\partial x} = -2.4 \times 0.1 = -0.24$$
$$\frac{\partial L}{\partial w_0} = \frac{\partial L}{\partial \text{op}_1} \times \frac{\partial \text{op}_1}{\partial w_0} = -2.4 \times 2 = -4.8$$

$\rightarrow$ don't have to calc. derivatives WRT const.

$\rightarrow$ learning rate is important. We don't use full grad. (-4.8) instead we use it partially / fraction of it.

gradient descent: After computing all gradients, update steps take place for learnable parameters.

Which params are updated in above ex?

- optimization algo is (mini-batch) gradient descent

update given by

$$w_0 = w_0 - \eta \frac{\partial L}{\partial w_0} ; w_1 = w_1 - \eta \frac{\partial L}{\partial w_1}$$

Process of FWD & BCKWD pass & param update is iterative until convergence. What is convergence? $\rightarrow$ either $L=0$ or w_0, w_1 don't change too much

with Pytorch gradients are computed automatically with automatic differentiation. (auto diff / autograd).

Programming new layers involves defining fwd & bckwd pass/step. With autodiff only fwd step is necessary. w/o autodiff we need to calc. numerical gradients for controlling backward steps gradients.

$\rightarrow$ generally i/p is a vector

$\rightarrow$ Jacobian matrix is then computed for all variables of an operation.

$\rightarrow$ grad. descent is applied on the same way on vectors.

Jacobian matrix for function $f: \mathbb{R}^4 \rightarrow \mathbb{R}^3$ with i/p x & o/p z .

derivative of o/p WRT each i/p is jacobian matrix.

$$J = \begin{bmatrix} \frac{\partial z_1}{\partial x_1} & \frac{\partial z_1}{\partial x_2} & \frac{\partial z_1}{\partial x_3} & \frac{\partial z_1}{\partial x_4} \\ \frac{\partial z_2}{\partial x_1} & \frac{\partial z_2}{\partial x_2} & \frac{\partial z_2}{\partial x_3} & \frac{\partial z_2}{\partial x_4} \\ \frac{\partial z_3}{\partial x_1} & \frac{\partial z_3}{\partial x_2} & \frac{\partial z_3}{\partial x_3} & \frac{\partial z_3}{\partial x_4} \end{bmatrix}$$

Back prop and Vectorization

Main diff. of having vector i/p or o/p is on backward step.

Jacobian matrix has to be computed for one or more pairs of i/p. A o/p

$$\frac{\delta L}{\delta x} = \frac{\delta L}{\delta z} \frac{\delta z}{\delta x}$$



$$\frac{\delta L}{\delta y} = \frac{\delta L}{\delta z} \frac{\delta z}{\delta y}$$

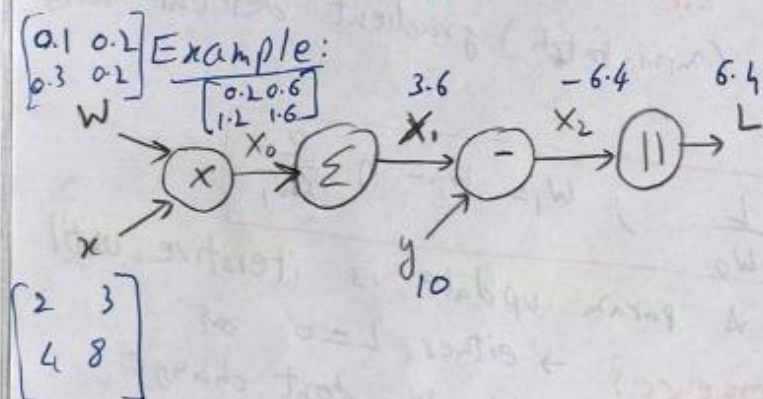
ex: if x & z are vectors then

$$\frac{\delta z}{\delta x} = \text{jacobian}$$

$$\frac{\delta z}{\delta y} = \text{jacobian}$$

Do we compute jacobian of loss? No, loss is a scalar

Example:



function: $f(x, W) = W \times x$
where $x \in \mathbb{R}^{n \times 1}$; $W \in \mathbb{R}^{n \times n}$

→ function in graph is **hadamard product**
includes loss function:

$$L = \left| \sum_{i=1}^n f(x, w) - y \right|$$

node(||) → $L = |x_2|$

$$\frac{\delta L}{\delta x_2} = \frac{x_2}{|x_2|} = -1 \text{ [local grad]} \quad \frac{\delta L}{\delta L} = 1 \rightarrow \text{upstream grad}$$

Node(-) → $x_2 = x_1 - y$

$$\frac{\delta x_2}{\delta x_1} = 1; \quad \frac{\delta x_2}{\delta y} = -1 \rightarrow \text{local grad.}$$

upstream grad → $\frac{\delta L}{\delta x_2}$

$$\frac{\delta L}{\delta x_1} = \frac{\delta L}{\delta x_2} \frac{\delta x_2}{\delta x_1} = -1 \times 1 = -1$$

Node(Σ)

$$x_1 = x_0[0,0] + x_0[0,1] + x_0[1,0] + x_0[1,1]$$

$$\frac{\delta x_1}{\delta x_0} = \begin{bmatrix} \frac{\delta x_1}{\delta x_0[0,0]} & \frac{\delta x_1}{\delta x_0[0,1]} \\ \frac{\delta x_1}{\delta x_0[1,0]} & \frac{\delta x_1}{\delta x_0[1,1]} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \rightarrow \text{[local grad]}$$

upstream grad → $\frac{\delta L}{\delta x_1} = -1$

$$\frac{\partial L}{\partial x_0} = \frac{\partial L}{\partial x_1} \frac{\partial x_1}{\partial x_0} = -1 \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} -1 & -1 \\ -1 & -1 \end{bmatrix}$$

Mode (x) $\rightarrow x_0 = w \times$ [element wise mul]

$$\frac{\partial x_0}{\partial w} = \begin{bmatrix} \frac{\partial x_0}{\partial w(0,0)} & \frac{\partial x_0}{\partial w(0,1)} \\ \frac{\partial x_0}{\partial w(1,0)} & \frac{\partial x_0}{\partial w(1,1)} \end{bmatrix} = \begin{bmatrix} 2 & 3 \\ 4 & 8 \end{bmatrix}$$

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial x_0} \frac{\partial x_0}{\partial w} = \begin{bmatrix} -1 & -1 \\ -1 & -1 \end{bmatrix} \begin{bmatrix} 2 & 3 \\ 4 & 8 \end{bmatrix} = \begin{bmatrix} -2 & -3 \\ -4 & -8 \end{bmatrix}$$

↗ element wise multiplication

finally $w = w - \eta \frac{\partial L}{\partial w}$

FWD $\rightarrow$ Bckwd $\rightarrow$ param update done iteratively until convergence

$\rightarrow$ passes work same for scalars & vectors.

$\rightarrow$ for vectors, gradients of a variable should have same dimension as the variable.

$\rightarrow$ operations can be described as layers / modules.

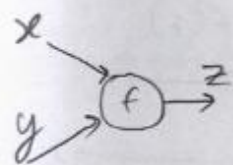
$\rightarrow$ it is required for modules / layers to be differentiable

$\rightarrow$ Popular modules: Relu, sigmoid, tanh, softmax, L2-loss & cross entropy.

Network Modules

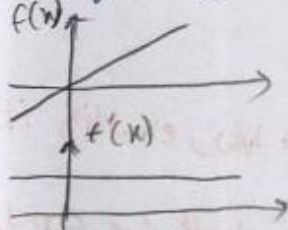
$f \rightarrow$ differentiable operations

$x, y, z \rightarrow$ can be scalar, vector, matrices or tensors.



1. Identity: $f(x) = x$; $f'(x) = 1$

Range: $(-\infty, \infty)$



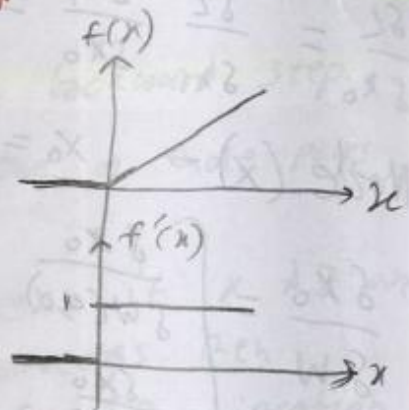
- Mostly used for regression problem.
- mostly found in the last layer of NN
- derivative is cont. & there is no vanishing grad.

2. ReLU Rectified Linear Unit what is motivation from nerves?

$$f(x) = \begin{cases} x & x \geq 0 \\ 0 & x < 0 \end{cases} \quad \text{Range } [0, \infty)$$

$$f'(x) = \begin{cases} 1 & x \geq 0 \\ 0 & x < 0 \end{cases} \quad \text{Range } \{0, 1\}$$

- derivative not cont. at $x=0$
- causes problems for non-tve values

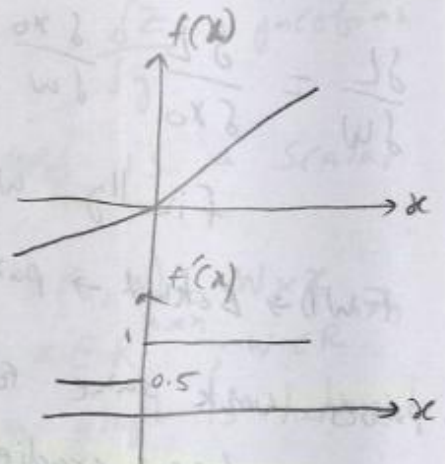


3. Leaky ReLU

$$f(x) = \begin{cases} x & x \geq 0 \\ 0.5x & x < 0 \end{cases} \quad \text{Range: } (-\infty, \infty)$$

$$f'(x) = \begin{cases} 1 & x \geq 0 \\ 0.5 & x < 0 \end{cases} \quad \text{Range: } \{0.5, 1\}$$

- allows -ve input to pass with small slope.
- derivative not cont. at 0
- common coefficient 0.01



Is Leaky ReLU good ^{activation} for regression?

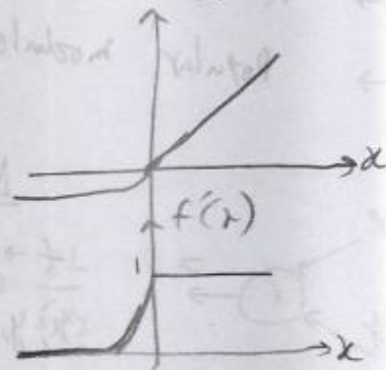
- No because o/p is not bounded

4. Exponential Linear Unit (ELU)

$$f(x) = \begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases} \quad \text{Range } \begin{cases} [-\infty, \infty) & \text{for } \alpha \geq 0 \\ [0, \infty) & \text{for } \alpha < 0 \end{cases}$$

$$f'(x) = \begin{cases} 1 & x \geq 0 \\ \alpha e^x & x < 0 \end{cases} \quad \text{range } [0, \alpha) \cup \{1\}$$

- converges faster than ReLU & delivers better perf.
- smoother than ReLU



Is ELU good for regression?

- Activation functions ^{like sigmoid/ReLU} are used to introduce non-linearity into h/w, enabling it to learn complex patterns in the data.
- Activation function determines whether/to what extent a neuron should be fired/active
- selected based on application.

5. Logistic sigmoid

$$f(x) = \frac{1}{1+e^{-x}}$$

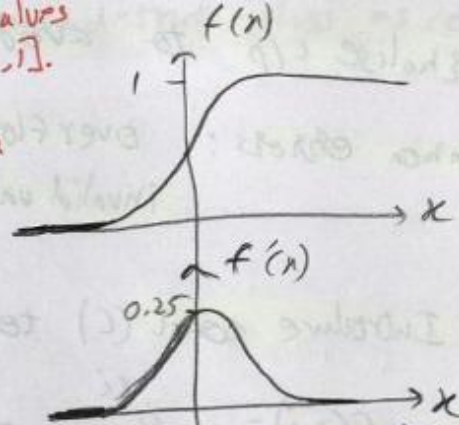
range $[0, 1]$

$$f'(x) = \frac{e^{-x}}{(1+e^{-x})^2} \text{ range } (0, 0.25]$$

→ derivative is cont. but vanishing gradients & saturation.

→ used for o/p activation on classification problems.

Squashes o/p values to o/w $[0, 1]$.
ideal for binary classification



stability in optimisation
Normalising effect
probability interpretation
of the func.?

6. Hyperbolic Tangent (tanh)

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

range $(-1, 1)$

$$f'(x) = 1 - f(x)^2 \text{ range } (0, 1]$$

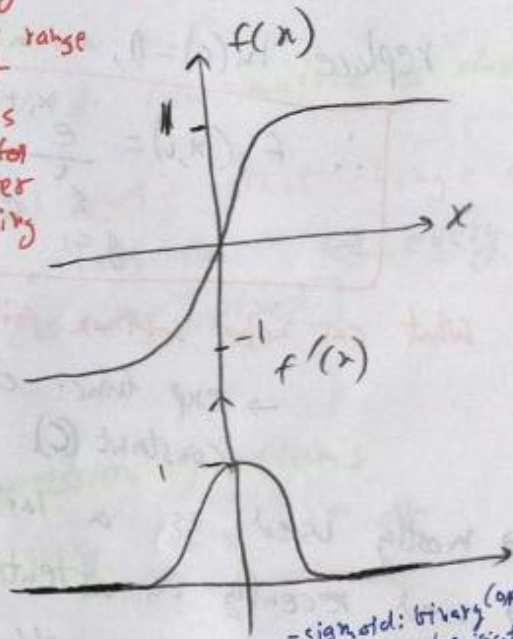
→ derivative is cont. but vanishing gradients & saturation.

→ used for o/p of classification prob.

What is diff from sigmoid?

+ve/-ve o/p. Symmetric around 0
zero centered o/p

maps up to range of
← centers data for better training



- sigmoid: binary (o/p layer)
classification
- tanh: hidden layers
(zero centered)

7. Softmax [Multiclass classification]

K-activations (poss. o/p) for K-class prediction.

$$f(x, i) = \frac{e^{x_i}}{\sum_{j=1}^K e^{x_j}} \quad ; \quad \sum_{j=1}^K f(x, j) = 1 \quad \text{range } [0, 1]$$

- i/p & o/p of softmax is represented by a vector. hence we compute Jacobian during backward step.

Jacobian during backward step.

$$\begin{bmatrix} \frac{\delta f^1}{\delta x_1} & \dots & \frac{\delta f^1}{\delta x_K} \\ \frac{\delta f^2}{\delta x_1} & \dots & \frac{\delta f^2}{\delta x_K} \\ \vdots & & \vdots \\ \frac{\delta f^K}{\delta x_1} & \dots & \frac{\delta f^K}{\delta x_K} \end{bmatrix}$$

$$\therefore \frac{\delta f^i}{\delta x_j} = \begin{cases} f(x, i) (1 - f(x, i)) & i=j \\ -f(x, i) f(x, j) & i \neq j \end{cases}$$

- normalise i/p to avoid too small/large numbers at the exponent.
- common errors: overflow in exp! Runtime Warning.
invalid value encountered in true divide ^{return} ~~value~~: RT ~~Warn~~ Warning
@np.sum(exp)

Introduce const. (C) to ensure it doesn't go out of bound

$$f(x, i) = \frac{e^{x_i}}{\sum_{j=1}^K e^{x_j}} = \frac{C e^{x_i}}{\sum_{j=1}^K C e^{x_j}} = \frac{e^{\ln(C) x_i}}{\sum_{j=1}^K e^{\ln(C) x_j}} = \frac{e^{x_i + \ln(C)}}{\sum_{j=1}^K e^{x_j + \ln(C)}}$$

replace $\ln(C) = 0$, an arbitrary const.

$$\therefore f(x, i) = \frac{e^{x_i + 0}}{\sum_{j=1}^K e^{x_j + 0}}$$

finally we choose

$$0 = -\max(x)$$

→ shifts the i/p closer to zero

What can make softmax fail when we have lot of classes?

→ exp func. can cause instabilities, hence we introduce constant (C).

- softmax can be combined with cross entropy for multi-class classification problems. x-entropy acts like loss function.

→ mostly used as a last module of a n/w.

→ used recently in attention mechanism as well.

→ i/p to softmax is called logits = non-normalised probabilities.

→ o/p of softmax is a probability distribution.

Cross Entropy.

measures dist. b/w 2 probability distributions.

1 distribution → o/p of softmax

other distr. → ground truth.

ground truth is one-hot vector that is defined as K-element vector
all its elements are zero other than correct class that is set to 1.

$$CE(p, q) = -\sum_k p(k) \log(q(k)); p, q \rightarrow \text{prob. distributions.}$$

y → ground truth; $p(k) \rightarrow$ prob. predicted by model / softmax o/p

$$CE(y, p) = -\sum_{k=1}^K y(k) \log(p(k)); y(k) \rightarrow \text{ground truth prob.}, p(k) \rightarrow \text{predicted prob.}$$

$y(k) = 1$ only for correct class hence,

$$CE(y, p) = -\log(p(y)); y \rightarrow \text{ground-truth class}$$

- considering only p as variable & ground-truth class as const

$$CE(p) = -\log(p_y)$$
- i/p $\rightarrow K$ -predictions } i/p A o/p of CE.
o/p $\rightarrow$ scalar
- Jacobian for backward step.

$$\begin{bmatrix} \frac{\partial CE}{\partial p_1} & \dots & \frac{\partial CE}{\partial p_K} \end{bmatrix}$$
but since it is evaluated only for correct class, it becomes

$$\begin{bmatrix} 0 & \dots & \frac{\partial CE}{\partial p_y} & 0 & \dots \end{bmatrix}$$
- combination of CE with softmax = std. for multiclass classification problem & is efficient.
How do we bring this to pixel level? what change on i/p or o/p? - we need a loss func. for every pixel [1 CE per pixel]

9. L1 and L2 loss

- most common loss functions for regression problems.
- target y of prediction x can vary from set of values to matrices & tensors.

L1 loss

- minimises absolute diff b/w prediction & target

$$L = \sum_{i=1}^K |y_i - x_i| ; K \rightarrow \text{o/p dimension}$$

L2 loss

- minimises squared diff. between prediction & target

$$L = \sum_{i=1}^K (y_i - x_i)^2 ; K \rightarrow \text{o/p dim.}$$

- can have $\sqrt{\quad}$ in addition. (L2-norm)

Ch-4 Optimisation

- in ML it differs from traditional optim. Traditional optim. ~~what we~~ actually minimise.
- in ML we care about some perf. metric on test set.
- but we don't optimise directly for metric. It is often not tractable / differentiable (F1-score / PR curve)
- min. other loss func. to improve perf. metric as well.
- Why use grad descent? → faster convergence
→ reach local minima.

in supervised learning:

$$J(w) = E_{(x,y) \sim \hat{p}_{data}} L(f(x;w), y)$$

↓ cost function

↓ empirical data distr. (training data)

↓ prediction function

↑ loss function

↑ input

↑ label

if we have access to data generation distribution p_{data} , cost becomes

$$J^*(w) = E_{(x,y) \sim p_{data}} L(f(x;w), y) \rightarrow \text{ideal cost func. due to lack of access to } p_{data}$$

Measures expected generalization error i.e. usually called risk.

Empirical Risk Min:

In ML goal is to minimise this risk $J^*(w)$.

but due to lack of p_{data} , it is not possible.

Hence we use $J(w)$, which is re-written as

$$J(w) = E_{(x,y) \sim \hat{p}_{data}} L(f(x;w), y) = \frac{1}{n} \sum_{i=1}^n L(f(x_i;w), y_i)$$

Training using ML algorithm results in minimizing the average error in the training set. This is known as empirical risk minimization.

We hope finally min. empirical risk = risk minimization.

Problems

→ disadv of empirical risk min = model can overfit to training data / memorise training data

→ few loss functions (L) where empirical risk minimisation not feasible due to poor gradients Ex (Hinge loss & 0-1 loss)

$$L(i,j) = \begin{cases} 0 & i=j \\ 1 & \text{otherwise} \end{cases}$$

sol: Use surrogate loss functions

Surrogate loss

0-1 loss for 2 class prob. is not right choice for neural networks.

Sol: optimise surrogate loss

-ve log-likelihood (NLL) of correct class is instead used for binary classification problem.

NLL $\rightarrow$ estimates the conditional class probability $\equiv$ least classification error (ie. risk)

$\rightarrow$ in some cases surrogate loss has better properties than loss.
(Ex: 0-1 v/s NLL)

Criterion for completion

pure optim
 $\rightarrow$ loss func. is minimised
 $\rightarrow$ iterate until gradients $\rightarrow 0$

ML algo.

$\rightarrow$ optim ends when convergence is reached
 $\rightarrow$ gradients might still exist but we stop if conv. is reached.

How do we observe convergence?

- $\rightarrow$ training loss
- $\rightarrow$ validation loss

Gradient Descent

$\rightarrow$ main optimisation algorithm for neural networks

Pipeline 1) training set $\{ \{x_1, y_1\} \dots \{x_n, y_n\} \}$

2) learn mapping function $f: X \rightarrow Y$, parametrized by w .

3) Define loss function L & then use back prop. to calc. gradients of w (parameters) & use grad. descent for updating parameters.

$w^* = \underset{w}{\operatorname{argmin}} L(f(x; w), y) \rightarrow$ find w that minimises the loss L .

Algorithm:

$\bullet$ start from an initial point [i.e. set of params] & take step opposite to direction of gradient.

(η) Learning Rate (LR) influences/regulates influence of gradient step.

Iteratively step until convergence: i.e. fwd pass $\rightarrow$ loss $\rightarrow$ back-prop $\rightarrow$ grad. descent

$$w = w - \eta \nabla_w L(f(x; w), y)$$

3 variants

1. Batch grad descent: $L(w) \rightarrow$ loss w.r.t parameters is computed on whole training set.

$$w = w - \eta \nabla_w L(f(x; w), y)$$

2. Stochastic grad descent: param update done for each sample separately.

$$w = w - \eta \nabla_w L(f(x_i; w), y_i)$$

3. Mini-batch grad. descent: perform gradient update for every mini-batch of n samples.

$$W = W - \eta \nabla_W L(f(x_{(i:i+n)}, W), y_{(i:i+n)})$$

Mini-batch version usually referred to as Stochastic-Gradient-Descent (SGD)

Challenges with grad descent

→ Neural network contain nonlinearities. As a result they have highly non-convex loss func.

→ Based on initial point, danger of getting stuck at local min, where $\text{grad.} = 0$.

good local minimas do exist in neural networks

→ Saddle points: optim. gets stuck.

It is point b/w 2 mountains. Local max for one dimension and local min for the other.

inference: similar sol. with diff. local minima possible, with same performance.

Choosing right learning rate not trivial.

→ Low LR can lead to slow convergence

→ large LR hampers convergence, loss func. can fluctuate around minima and even diverge.

→ LR is same for all parameters, but learned features should be treated differently based on their sparsity.

→ adjusting LR is necessary. large LR required only at beginning. update rule necessary.

[1 LR might not be enough, 1 LR for every parameter might be req.]

→ shuffling data helps neural-net. to not memorise data.

local min. A saddle points: sol → mini-batch SGD & add noise to labels. in addition shuffling train data at every epoch helps.

these 3 tricks are necessary & useful but don't always help to escape from saddle points.

To escape from these saddle points we use other variants of optim. algos.

[All 1st order algos]

Gradient descent variants

1. SGD with momentum
2. Nesterov accelerated gradient (NAG)

3. Adagrad
4. Ada delta
5. RMS prop.
6. Adam

SGD with momentum: helps avoid getting stuck in minimisation

- momentum accelerates convergence by reducing oscillations.
- accumulates past gradients to moderate direction of current gradient.
- aim of momentum is to increase over-time & add more velocity.

$u \rightarrow$ velocity $\rightarrow$ decides how much to use from accumulating var.

$$u \leftarrow \alpha u - \eta \nabla_w \left(\frac{1}{m} \sum_{i=1}^m L(f(x^i; w), y^i) \right)$$

std. back prop. gradients
vel exponentially decays based on -ve gradient of mini-batch with m -samples

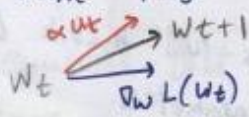
→ $\alpha \in [0, 1]$, is usually set to 0.9; η depends on networks, tasks & data

$$w \leftarrow w + u \quad [\text{update step}]$$

In general, vel. increases in dim. where gradients point in same direction, decreases when point in different directions.

Momentum from Physics & Newton's law of motion.

$$p = mv; \text{ here we assume unit mass } \Rightarrow p = v [\text{momentum} = \text{vel.}]$$



NAG

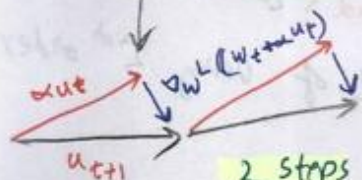
- similar to classic momentum
- diff. is gradient is evaluated after applying velocity.

$$u \leftarrow \alpha u - \eta \nabla_w \left(\frac{1}{m} \sum_{i=1}^m L(f(x^i; w + u), y^i) \right)$$

update weights to calc. new grad.
x velocity $\rightarrow$ apply vel. to w

$$w \leftarrow w + u \quad [\text{update step}]$$

→ evaluation of grad. at $w + u$ can be interpreted as correction to std. momentum.



2 steps: NAG 1st takes a step (red vec) based on accumulated grad. A 2nd estimates grad. at landed pos. to make the correction (blue vec).

Adagrad

- uses 1 rate per parameter
- often some parameters are more sensitive to gradient updates than others to avoid perf. drops & convergence difficulties gradient influence should not be the same for all parameters.
- can update network params individually based on their sensitivity
- Adaptive learning algs take care of updating parameters individually.

Algorithm:

- LR is adapted to lower values for frequently updated parameters.
- LR is adapted to higher values for infrequently updated parameters.

Considering a W layer, i.e. set of parameters as features. same applies to frequently updated features have lower LR & so on.

How can we measure update freq. for each param?

$$g = \nabla_W \left(\frac{1}{m} \sum_{i=1}^m L(f(x^i, w), y^i) \right)$$

→ gradient

$$G = G + g \otimes g$$

↗ element wise multiplication

→ accumulated squared gradient

$$W \leftarrow W - \frac{\eta}{\sqrt{\epsilon + G}} \odot g$$

→ update step

↖ elementwise square-root

observations:

- G is a diagonal matrix. can be treated as full matrix but computations cost is large even for small amount of parameters.
- gradients accumulated over time for each training iteration.
- ϵ , is small value added for stability
- $\sqrt{\cdot}$, is applied element-wise
- Adv: automatic tuning of LR. commonly $LR = 0.01$
- LR is constantly scaled by accumulated gradients, LR becomes smaller over time. → what is limitation of Adagrad? [almost no LR]
- ↳ nice property of 2nd order methods, without the cost of using 2nd order method such as Newton's method.

Adadelta [attention of Adagrad]

- allows to adapt LR of each parameter (i.e. per dimension) independently.
- motivation: to avoid insistently reducing LR to zero.
- Adagrad has a global LR that requires some tuning.
- has small overhead compared to SGD, similar to Adagrad.

idea: Accumulate past gradients over a time window.

- accumulation is exponentially decaying average of the squared gradients (over time window)

- Moving average $E[g^2]$ depends only on its prev. average & current gradient. similar to SGD where decay const. γ controls contribution of prev. avg.

$$E[g^2] = \gamma E[g^2] + (1-\gamma)g^2,$$

where $g = \nabla_w \left(\frac{1}{m} \sum_{i=1}^m L(f(x^i, w), y^i) \right)$

- $\gamma = 0.9$ usually (as with momentum)

$$w \leftarrow w - \frac{\eta}{\sqrt{\epsilon + E[g^2]}} \odot g,$$

add moving avg to param update to obtain this eqn.

ignoring denominator $\Rightarrow \text{RMS}[g] = \sqrt{E[g^2]}$

$$\Rightarrow w \leftarrow w - \frac{\eta}{\epsilon + \text{RMS}[g]} \odot g$$

Units: It is required that gradient update has same units as the parameters.

- use hypothetical units as we don't use any units.

- lack of updating the same units is done by SGD, momentum & Adagrad.

Ex: SGD & Momentum.

$$\nabla_w L = \frac{\partial L}{\partial w} = \frac{1}{w \text{ units}}$$

Adadelta

$$\Delta w_t = \frac{\eta}{\epsilon + \text{RMS}[g]} \odot g$$

$$\Rightarrow \Delta w_t = \frac{\frac{\partial L}{\partial w}}{\frac{\partial^2 L}{\partial w^2}} \Rightarrow \frac{1}{\frac{\partial^2 L}{\partial w^2}} = \frac{\Delta w}{\frac{\partial L}{\partial w}}$$

ΔW is not known. Assume cont. curvature & use term from last step.

$$\Delta W_t = \frac{\Delta W}{\frac{\delta L}{\delta W}} = - \frac{\text{RMS}[\Delta W]_{t-1}}{\text{RMS}[g]_t} \odot g_t$$

→ note: global LR(η) has vanished.

→ units are the same now

$$W \leftarrow W - \frac{\text{RMS}[\Delta W]_{t-1}}{\text{RMS}[g]_t} \odot g_t$$

RMS also integrates ϵ for numerical stability.
Most imp. for 1st iter.

→ optimisation is robust to sudden gradient changes, since denominator will increase & thus LR will decrease.

RMS prop

motivation: same as Adadelta, i.e. to improve Adagrad.

→ LR may need to be very small to reach a convex structure, where the sol. lies.

$$g = \nabla W \left(\frac{1}{m} \sum_{i=1}^m L(f(x_i, g), y_i) \right) \rightarrow \text{compute grad}$$

$$G = \alpha G + (1 - \alpha) g \odot g \rightarrow \text{accumulate squared grad.}$$

$$W \leftarrow W - \frac{\eta}{\sqrt{\epsilon + G}} \odot g \rightarrow \text{update step.}$$

→ $\alpha = 0.9$ [similar to SGD with momentum]

→ Accumulation of squared gradient is same as Adadelta. Use a moving avg. again.

$$E[g^2] = \gamma E[g^2] + (1 - \gamma) g^2$$

$$\Rightarrow \Delta W = - \frac{\eta}{\epsilon + \text{RMS}[g^2]} \odot g$$

→ diff. from Adadelta is doesn't consider concept of working on same hypothetical units.

→ RMS prop also retains same LR $\eta = 0.001$

Adam [Adaptive moment estimation]

→ use past squared gradients until now.

→ Adam stores, in addition, past grad. similar to momentum algorithms (i.e. velocity). 2 moving averages to compute

→ 1st order grad. based optim. algo.

→ more memory requirements compared to Adagrad, Adadelta, RMS prop.

→ suited for noisy and/or sparse gradients.

→ relies on 2 moments

mean (1st moment) → $m_t = \beta_1 m_{t-1} + (1 - \beta_1) g$
uncentered (2nd moment) → $u_t = \beta_2 u_{t-1} + (1 - \beta_2) g^2$
variance

hyper-params

$\beta_1, \beta_2 \in [0, 1)$ control exponential decay rate of m_t & u_t .

→ 2 moments are initialised with zeros. (creating bias towards zero estimates. (especially when $\beta_{1,2}$ are close to 0) that needs to be removed.

→ bias corrected estimates

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$
$$\hat{u}_t = \frac{u_t}{1 - \beta_2^t}$$

Why is bias important?
How do we derive bias corrected estimates?

Derivation: [we look at 1st momentum]

$$\textcircled{*} m_t = \beta_1 m_{t-1} + (1 - \beta_1) g \Rightarrow m_t = (1 - \beta_1) \sum_{i=1}^t \beta_1^{t-i} g_i$$

we get above eq. from expanding $\textcircled{*}$

$$m_0 = 0$$

$$m_1 = \beta_1 m_0 + (1 - \beta_1) g_1 = (1 - \beta_1) g_1$$

$$m_2 = \beta_1 m_1 + (1 - \beta_1) g_2 = \beta_1 (1 - \beta_1) g_1 + (1 - \beta_1) g_2$$

$$m_3 = \beta_1 m_2 + (1 - \beta_1) g_3 = \dots$$

Now examining expected values of m & how it relates to true 1st moment,

$$\begin{aligned}
 E[m_t] &= E\left[(1-\beta_1) \sum_{i=1}^t \beta_1^{t-i} g_i\right] \\
 &= E[g_t] (1-\beta_1) \sum_{i=1}^t \beta_1^{t-i} + \zeta \\
 &= E[g_t] (1-\beta_1^t) + \zeta ; \quad \zeta \rightarrow \text{approximation error.}
 \end{aligned}$$

Note

$$\begin{aligned}
 (1-\beta_1^t) \sum_{i=1}^t \beta_1^{t-i} &= \beta_1^t (1-\beta_1) \sum_{i=1}^t \beta_1^{-i} \\
 &= \beta_1^t (1-\beta_1) \frac{\beta_1^{-1} (1-\beta_1^{-t})}{1-\beta_1^{-1}} = (1-\beta_1) \frac{\beta_1^{t-1} (1-\beta_1^{-t})}{\beta_1^{-1}} = (1-\beta_1^t)
 \end{aligned}$$

After few thousand iter. it reaches value 1.
 → True 1st moment $E[g_t]$ is scaled by $(1-\beta_1^t)$, due to initialization of m vector with 0 s.

To correct this scale we divide it by $(1-\beta_1^t)$

$$W \leftarrow W_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{u}_t} + \epsilon}$$

Recommended
 $\epsilon = 10^{-8}$

$\beta_1 = 0.9, \beta_2 = 0.999$

Observations

- faster convergence than SGD with momentum, Adagrad, RMSprop, Adadelta.
- can converge to worse solutions.
- combined with Nesterov momentum, to give more stable sol.

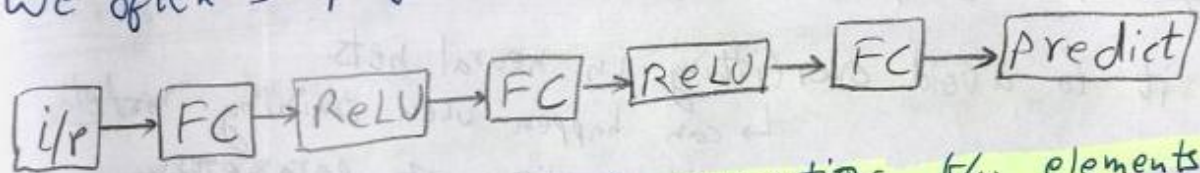
Choice of optimiser

sparse data → optimiser with adaptive LR.

Fast conv. → " " " "

Big amount of training data → SGD with / w/o momentum

We often simplify neural network representation using blocks.



FC: refers to all-with-all connections b/w elements of the 2 blocks. [Fully connected layer]

A neuron is a combination of FC & activation [ReLU]

Now we look at several aspects of training a NN.

- LR
- Regularisation
- dropout
- initialisation
- normalising i/p & o/p
- batch normalization.

Learning rate: (LR)

- diff. optimizations adapt LR of each parameter individually / discard the global LR.
- When we don't have a global LR we need a rule to adjust it.
 - as training converges LR should be lowered to avoid jumps around the local minima.
 - right LR leads to faster convergence.

strategies

1. **Constant**: LR remains same for all epochs.
could cause slow convergence.
will work if LR is correctly chosen.
2. **Step decay**: decreases every k epoch by a factor.
Ex: can be decreased by a factor of 10 every 100 epochs.
we will end up with 0.01, 0.001, 0.0001 for 300 train. epochs.
3. **Inverse decay**: formulated as $\eta_t = \frac{\eta_0}{1+ct}$; $t \rightarrow$ iteration
 $c \rightarrow$ constant
4. **Exponential decay**: $\eta_t = \frac{\eta_0}{e^{ct}}$

[step decay most commonly used & it doesn't require a lot of tuning]

All above steps 2, 3, 4 monotonically decrease LR, however value of initial LR is important. To avoid looking for proper LR we consider:

5. **Cyclical Learning rate scheme**: Define upper & lower bound of LR, i.e. base & max. LR. We let learning to oscillate b/w the 2 bounds over training by changing for each step size. $\text{stepsize} = \frac{\text{no of } t}{\text{iter}}$

Ch5 Regularization: Imposes a preference to the solution.

→ we use it to avoid overfitting in neural nets.
↳ can happen when training models with millions of parameters.

→ Additional minimization term next to loss function,
↳ hyper param for tuning contribution of regulariser
↓
Regularization

$$\text{eqn 1} \quad w^* = \arg \min_w L(f(x, w), y) + \lambda R(w)$$

2 - Popular types of regularization to instantiate R ,

1. L2-regularization

$$w^* = \arg \min_w \frac{1}{m} \sum_{i=1}^m L(f(x^i; w), y^i) + \frac{\lambda}{2} \|w\|^2$$

Regularisation aims to reduce test error.

Prevents overfitting

Improves generalisation

For gradient update, we have

$$w = w - \eta \nabla_w \left(\frac{1}{m} \sum_{i=1}^m L(f(x^i; w), y^i) + \frac{\lambda}{2} \|w\|^2 \right)$$

$$\Rightarrow w = w - \eta \nabla_w \frac{1}{m} \sum_{i=1}^m L(f(x^i; w), y^i) - \eta \lambda w$$

$$\Rightarrow w = \underbrace{(1 - \eta \lambda)}_{\text{weight decay}} w - \eta \nabla_w \frac{1}{m} \sum_{i=1}^m L(f(x^i; w), y^i)$$

weight decay. λ is usually 0.1 or 0.01.

2. L1-regularization

$$w^* = \arg \min_w \frac{1}{m} \sum_{i=1}^m L(f(x^i; w), y^i) + \lambda \|w\|$$

For grad. update:

$$w = w - \eta \nabla_w \left(\frac{1}{m} \sum_{i=1}^m L(f(x^i; w), y^i) + \lambda \|w\| \right)$$

$$\Rightarrow w = w - \eta \nabla_w \frac{1}{m} \sum_{i=1}^m L(f(x^i; w), y^i) - \eta \lambda \frac{w}{\|w\|}$$

$$\Rightarrow w = \left(1 - \frac{\eta \lambda}{\|w\|}\right) w - \eta \nabla_w \frac{1}{m} \sum_{i=1}^m L(f(x^i; w), y^i)$$

Regularisation forces weights to become 0, i.e. sparse weights.

Regularisation can be explicitly added to objective function in eqn 1

Early stopping: is another type of regularisation, it doesn't have an explicit form.

- refers to termination of training before over-fitting starts.
- for this we observe training & validation loss/error over time & stop training at the right time.
- stop when validation error starts increasing.

Data augmentation: another form of regularisation that aims for invariance to data transformations.

- labels can be augmented too by adding small amounts of noise, when we deal with regression problems.
- popular on structured data such as images & videos.
Ex: rotation, crop, scale, flip & color adaption.
- can also be applied to 3D data such as point clouds & voxel representations. 3D coordinates can be shifted.
- In NLP it is common to exchange words with related words. or other ways of augmentation can be to change tense/subject of a sentence.

Is this technique useful for classification problems?

Max Norm Constraints: it is implicit regularisation

- we can force norm of the parameters not to exceed some constant.

$$\|w\|^2 \leq c ; c \in \mathbb{R}$$

value of c depends on number of network parameters.

- related to max norm constraints, we can clip the gradients of the parameters. This is done to prevent network parameters from exploding. However we can use it to impose a preference towards the update of the parameters.

[less commonly used]

Dropout: idea is to randomly shut off units (neurons), i.e. activation functions & their connections during training. This prevents from co-adapting.

What is co-adaptation?

- all network units are optimized for same objective, i.e. to minimise loss.
- consequently, derivatives of error-prone units will adapt the units with correct predictions too, complex adaptations b/w units will be created.
- this is interpreted as over-fitting as co-adaptations don't generalise to unseen data.

Dropout prevents co-adaptations by creating randomly different network structures.

- Neural net trained with dropout can be seen as an ensemble of models. This is because multiple networks will be implicitly trained due to dropout layer.

Working during training:

- drops out neurons with probability p coming from Bernoulli's distribution.
- rest of neurons with prob. $q=1-p$ remain active.

Given x as an o/p of activation function, dropout applies as an extra layer with i/p x .

$$f(x; D) = D \odot x \quad ; \quad D_i = \{0, 1\}$$

Moreover we have

$$P(D_i=0) = p \quad \text{A} \quad P(D_i=1) = 1-p$$

p is commonly 0.5, i.e. 50% of units are dropped out.

During training: Ensemble of networks is trained based on dropout.

At inference (test-time)

- we can't use complete network as it is.
- we need to scale activations in order to emulate training.
- hence all activations are scaled with prob. q during testing.
- our implementation should have flag to switch from train to test mode.

[has been used to add randomness in GANs [Generative Adversarial network]]

It has been used as Bayesian approximation of uncertainty.

Inverted Dropout: variant of dropout to simplify its implementation

- activations already scaled during training to avoid network modifications during test.
- this means we can scale all activations by $\frac{1}{q}$
- Adv: we define the network only once. During test, we only need to modify the dropout probability.

observations:

- dropout used to prevent over-fitting
- inverted dropout implemented in practice
- For single neuron dropout is applied by sampling from Bernoulli distribution.
- For multiple neurons we can sample from a binomial distribution.
- Dropout is used traditionally on FC layers.

Parameter initialisation

Optimisation algorithms require initial point to start learning. The choice of this parameter initialisation affects the convergence of deep learning algorithm.

- bad init. ^{may} leads to numerical difficulties & convergence failure
- init. affects rate of convergence. convergence of algorithm is faster with good init.
- init. influences generalization error as well. Test error can be reduced with good init.
- Most widely used init. are based on heuristics, due to lack of understanding of how deep learning works.

Requirements to define init. strategy:

1. Weights need to be around zero for symmetric functions
2. weights need to be large enough to produce useful gradients. smaller gradients hinder the optimization.
3. Input and output variance of the units should be same.
4. Consider non-linearities as activation functions too.
5. If data affects the weights.

Symmetry-breaking initialization

idea: each unit has different parameters so that it approximate a different function.

→ 2 units with same i/p must have diff. parameters, otherwise, they will be updated in same way leading to deterministic o/p.

→ Having diff. parameters for each unit leads to idea of random param initialization.

→ Alternatively one can search for set of functions which are diff. from each other (ex: Gram-Schmidt orthogonalization), but it is time consuming.

→ simple randomization of parameters is sufficient.

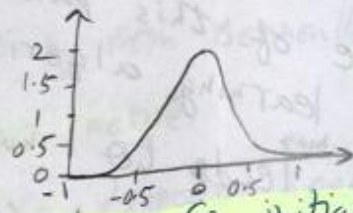
→ bias term is usually initialised to some constant.

2 std. distr. to initialise network param

- Gaussian distr.
- Uniform distr.

1. Gaussian Distribution: Initialization

$$P(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$



→ most common way to sample random values for initialisation, requiring only mean and standard deviation.

→ generates unique units

→ computational effort is minimal

→ initialization usually has zero mean & std. deviation 1.

→ Biases are usually set to zeros/1s.

2. Uniform distribution: Initialization

$$P(x) = \begin{cases} 0 & x < \alpha \\ \frac{1}{\beta - \alpha} & \alpha \leq x \leq \beta \\ 0 & x > \beta \end{cases}$$



sometimes important to scale this distr. values based on i.o. of model parameters. Hence more precise initialization can be achieved with Global Training

→ we need to decide interval

→ sampling process is same as with gaussian distr.

→ range of sampled values can be from 0 to 1

→ biases set usually to zero or 1

Scale of initialisation

- scale of values is important factor of initialization.
- the distribution parameters of gaussian/uniform have less influence on the convergence.
- scaling parameters to have large values will produce significantly different units.

This is what we aim to have.

Signal will have strong response in FWD step & will produce a large gradient at backward step.

- on otherhand, large initial parameter values have disadvantage:
 - gradients can explode
 - activation functions can saturate

- training recurrent neural networks with large weights is not easy.

Recall the sigmoid & hyperbolic tangent activation functions.

- strategy of large initial weights contradicts idea of regularisation, where weights are pushed to ~~zero~~ have small values. During optimisation, regularisation encourages parameters to be small and sparse.

- gradient descent algo. incrementally updates parameters based on previous state. This means final sol. can be close to initialization, hence, proper init. is important.

- choosing right scale can be done heuristically. However, it is dataset dependent & time consuming. We need an automatic way to scale/initialise parameters.

→ discovery by trial & error

Glorot initialisation [works well for MLP] / Xavier initialisation

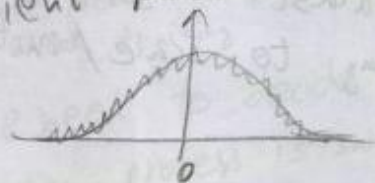
- motivation for this work was to understand why grad. descent performed poorly with random initialisation.

given a FC layer of an MLP, normalised initialisation for each parameter $\Rightarrow$

$$w_{i,j} \sim U\left(-\sqrt{\frac{6}{d_{in}+d_{out}}}, \sqrt{\frac{6}{d_{in}+d_{out}}}\right) \quad \text{--- (1h)}$$

$\downarrow$ $\downarrow$ $\downarrow$
Uniform No. of No. of
distr. i/p o/p

- $\rightarrow$ Uses uniform distr. to sample parameter values.
 - range of values is defined by no. of i/p & o/p params
 - $\rightarrow$ this initialization is useful for networks with tanh activation functions.
 - $\rightarrow$ params have similar ranges of gradient values.
 - $\rightarrow$ no units with same parameters
 - $\rightarrow$ motivation of the formula is based on representing neural networks by chain of matrix multiplications without non-linearity activations. although this assumption doesn't work in practice, init. rule works.
 - $\rightarrow$ ensures init values are ~~not~~ ^{neither} concentrated too much around zero nor spread out too widely
- Which fig. represents Glorot initialisation?
- $\rightarrow$ gradients are more evenly distributed across layers, avoiding vanishing/exploding gradient problem.



General form:

consider a single unit given as, $y = w_1 x_1 + \dots + w_d x_d$; $x \rightarrow$ i/p
 $w \rightarrow$ parameters

assuming x & w are independent params, (d in total)
variance is, $\text{Var}(y) = \text{Var}\left(\sum_{i=1}^d w_i x_i\right) = \sum_{i=1}^d \left(E[x_i]^2 \text{Var}(w_i) + E[w_i]^2 \text{Var}(x_i) + \text{Var}(x_i) \text{Var}(w_i) \right)$

if we assume samples are drawn from a Gaussian distr. with zero mean for both x & w , 1st 2 terms can be cancelled.

$$\Rightarrow \text{Var}(y) = \text{Var}\left(\sum_{i=1}^d w_i x_i\right) = \sum_{i=1}^d \text{Var}(w_i) \text{Var}(x_i)$$

if we further assume x & w are identically distributed,

$$\Rightarrow \text{Var}(y) = \sum_{i=1}^d \text{Var}(w_i) \text{Var}(x_i) = d \text{Var}(x) \text{Var}(w)$$

$$\left[\because \text{for uncorrelated variables} \right. \\ \left. \text{Var}\left(\sum_{i=1}^d x_i\right) = \sum_{i=1}^d \text{Var}(x_i) \right]$$

→ we want variance of o/p y = var of i/p x .

$$\Rightarrow \text{Var}(w) = \frac{1}{d}, \text{ such that i/p \& o/p variance are equal, i.e.}$$

$$\text{Var}(y) = d \frac{1}{d} \text{Var}(x)$$

To sum up, we initialize weights from Gaussian distribution:

$$(19) \rightarrow w_{ij} \sim \mathcal{N}\left(0, \sqrt{\frac{1}{d}}\right); d \rightarrow \text{n.o. of inputs.} \\ \rightarrow \text{Glorot init. (general form)}$$

→ Unlike in (18), in (19) we have general rule that applies to any kind of neural network architecture.

→ used for MLP, conv. neural networks & recurrent networks in combination with non-linearities.

→ Not optimal for ReLU activation functions.

What is the problem / limitation of Glorot init. when network has ReLU activations?

↳ ReLU works only with +ve values

↳ Glorot init. is not effective since half of the parameters can be eliminated by ReLU.

→ we had assumed here "if the samples were from Gaussian distr. with 0 mean. But when using ReLU, 0 is not the mean. instead we have

$$E[x_i]^2 = \frac{1}{2} \text{Var}(y_{\text{previous}}) \quad \left[\because \text{relu grounds half of the -ve values} \right]$$

He / Kaiming initialization

$$w_{ij} \sim \mathcal{N}\left(0, \sqrt{\frac{2}{d}}\right)$$

→ Glorot & He are very similar

→ He is recommended for modern & very deep neural network architecture

Saxe Initialization

- initialization with random orthogonal matrices.
- SVD can be used on the weights for this purpose.
- additional scale/gain is introduced to adapt weights base on non-linear activation functions.
- increasing gain of network layer pushes activations to have higher response & larger gradients.
- has been shown that very deep neural networks can be trained when using the correct gain, even without orthonormal init.

LSUV initialization [~~Layer~~ Layer-sequential unit-variance]

- Extension of Saxe init.
 - 2 step process
 - 1) initialise with orthonormal matrices similar to Saxe init.
 - 2) scale weights to make variance = 1, based on estimated o/p variance of each layer
- This 2nd step requires to load mini-batch data & perform a fwd propagation step.

shows promising results but not as popular as He-init which is used by default for deep learning frameworks.
reason could be due to 2 step process requiring FWD step to complete init.

Choice of weight initialisation:

No guarantee of reaching good min. despite using above init approaches because → what are the conditions?
conditions which should be met during init. do not hold anymore after starting the training.
imposed conditions during init do not necessarily mean better generalization
init. can be problem specific. we might need to try diff. approaches for our data.

Input Normalization

Parameter & activation functions values should be within some range to assure good convergence.

same principle applies to network i/p as well.

Input data goes through 1st network layer, followed by activation.

Ex: image pixels or features such as height, speed, vol. can cause instabilities due to high range of feature values.

→ activation & gradient range give us an indication how to normalize the input data.

Rescaling [min-max normalisation]

Approach: center the data around zero.

How? :

- can scale data to $[0, 1]$ or $[-1, 1]$
- requires knowledge of min & max values for each i/p dimension in advance.
- we also need to compute extremal values using only the training data & assume that they represent the test set too.
- can have -ve impact when some of i/p dimensions have a range of values that significantly differ from the rest.
- it is important to not create correlations b/w data during i/p pre-processing. If it happens then it has -ve impact on optimization.
- sequential data is correlated by default & i/p normalization is often not helpful in this case.

Standardization (Unit Normalization)

[most common i/p normalisation]

Approach: center data around zero.

Assumption is data comes from a normal distribution with zero mean & std deviation one. 2 steps to reach it:

- 1) subtract mean from data
- 2) Divide data by standard deviation

For single i th dim. of i/p x :

$$x_i = \frac{x_i - \text{mean}(x, i)}{\text{std}(x, i)}$$

- it is similar to data-whitening where covariance matrix of the i/p data dimensions is converted to identity matrix given that data is uncorrelated (zero mean).

Input normalization

- Unit length: i/p feature vector can be scaled to have length one.
- useful when fusing ex: by concatenation, different modalities with large diff in range of values.
- Dimensionality reduction: aim is to reduce redundant info. and correlations. (ex pixels with same values)

How can we reduce ^{data} dimensionality?

disadvantage of unit length normalisation?

- in some cases ~~that~~ normalisation is harmful (ex: sequential data)
- takes place only for i/p data.
- After this point we don't control range of data values which means we can have large values that could -vely impact our neural network.
- sol to better control o/p of each network layer is to introduce an additional normalisation after a network operation.

Batch Normalisation

- activations can vanish while going deeper into layers.

Motivation: same as i/p normalisation, i.e. to standardise ~~the~~ data.

- data corresponds to some distr. that can change significantly within layers of a deep neural network.
- batch normalisation: aims to fix the internal covariate shift. This happens when data distr. fluctuates. ^{takes place in internal layers of the network.}
- data distr. ~~that~~ depends on layer weights that vary, i.e. covariate.
- the distr. can shift during training again & again.

- goal is to perform unit normalisation of intermediate layers i.e. **FC layers**.
- normalization is done prior to activation. In this way, i/p to activation will never cause an explosion/attenuation.
- i/p normalisation can use whole training set. This is not possible while training for computational reasons, which is why we rely on the m samples of the mini-batch.

Algorithm:

For each mini-batch of m -samples perform following steps:

1) compute mean $\mu_B = \frac{1}{m} \sum_{i=1}^m x_i$

2) compute variance $\text{var}_B = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2$

3) Normalise the i/p

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\text{var}_B + \epsilon}}$$

$x \rightarrow$ single dim. i/p
 $\epsilon \rightarrow$ small const. for numerical stability.

Scale & shift:

To relax our constraints about data range of values, we introduce a scaling variable γ & other shifting variable β . γ & β allow the network to modify range of values.

4) Scale & shift $y_i = \gamma \hat{x}_i + \beta$

γ, β are also learnable, optimised during training.

→ when having more parameters (or features for conv Nets) we perform batch normalization for each parameter, i.e. dimension, independently.

→ The μ_B, var_B can be learned during training as a moving avg. & then use in testing.

Why is it necessary to learn μ_B & var_B ?

Observations:

→ shift of batch normalisation = bias term.
Hence we can remove bias terms from our network when using batch-normalization.

During training we use mean & variance of the mini-batch.
What do we use at test time?

During training, we can compute exponentially weighted running average for mean & var. We can use these values during test.

→ it doesn't prevent internal covariate shift. But works well in practice.

→ Assumption is it makes optimisation smoother, resulting in more stable gradients & faster convergence. Stabilises optim. & leads to faster convergence.

→ recent approaches have shown that good init. is sufficient for training networks upto 10,000 layers w/o batch normalisation.

→ Batch norm. is performed after a fully-connected layer.

→ For FC layer it is performed for all parameters, while in a convolutional layer it is performed per feature channel.

Limitations

→ mini-batch size affects mean & variance computation

→ doesn't work well with RNNs.

Other techniques

The 3 follow from motivation of batch normalisation.

1) Layer Normalisation: μ & var computed along the channels, width & height axes (for convNets). The batch size doesn't affect the normalization.

Recommended for RNNs.

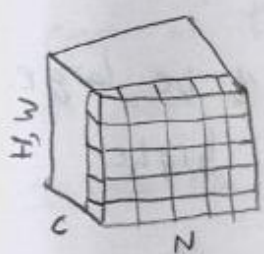
Not directly applicable to FC layers

2) Instance normalization:

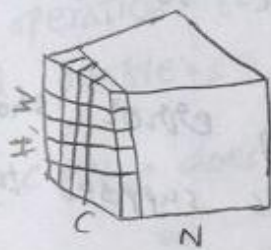
- μ & var computed along width & height axes for each sample & each channel (for ConvNets)
- Not directly applicable to FC layers.

3) Group norm.

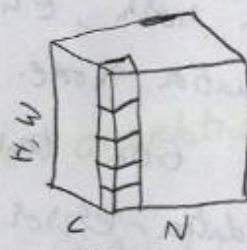
- μ & var computed for groups along the channels, width & height axes (for ConvNets)
- Not directly applicable to FC layers.



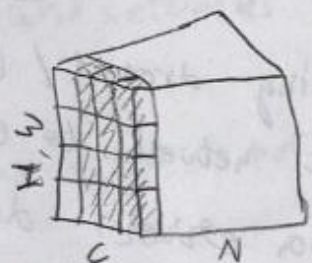
Batch Norm



Layer Norm



Instance norm



Group Norm

→ representations of a tensor

C → feature channels
W → width
H → height
N → mini batch size

Output Normalization

- normalization of i/p & intermediate data was helpful for better optim.
- optim can be better fitted by normalising the o/p too.

1) During regression, constrain to o/p to $[0, 1]$ or $[-1, 1]$.

This entails that we know extrema values for each dim. However in practice we use training data to obtain these values. But we have prob. if range of test data is different.

2) During classification there is no need for o/p normalization.

Batch norm. performed after FC layer but before activation.

Batch norm: across (N, H, W) for each channel C.

layer norm: (C, H, W) for each instance N

Instance norm: (H, W) for each channel C per instance N

group norm: (H, W) groups of channels within instance N

Hints for training Neural Net.

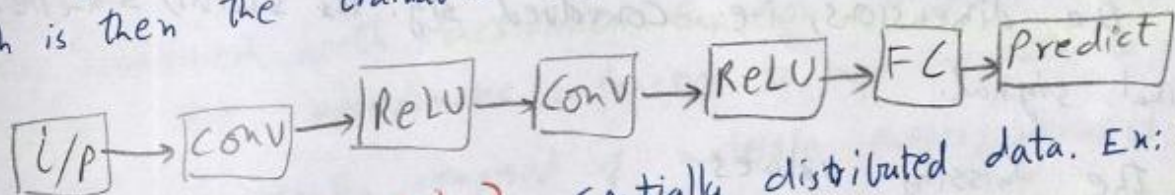
- Grad. check: compute gradients with numerical approach to ensure they are correct.
- Sanity check: Train with few samples.
If η doesn't overfit, there might be a bug.
Don't forget to remove regularization, dropout / batch norm.
- Use Regularization: at least L2 reg. should be employed.
Conditional dropout & batch norm help too.
- if using dropout / batch norm, ensure to execute the func. to set network to evaluation mode.
- Validation error: diff. from training error shouldn't be large.
Also validation error is supposed to be higher than training error.
- Use only He init for networks with ReLU activation.

Hints for convergence

- check correctness of gradients.
- " vanishing / exploding gradients
- " range of grad. values for each layer.
- debugging layer by layer can help to find what causes the problem

Ch-6 Convolutional Neural Networks

CNN is similar to previous architectures, where main difference is replacement of FC layers with conv. layers. Increase of classification we need atleast 1 FC layer at the end, which is then the trainable classifier.



What is structural data? Spatially distributed data. Ex: 1D, 2D images.

Conv. is the operation that made deep neural networks to scale on real world problems

convolution: [CNN → does cross correlation as filter is not flipped]

$$f * g = \int_{-\infty}^{\infty} f(\tau) g(t-\tau) d\tau$$

conv. b/w f & g signals results in a new signal with modified shape/ response.

$f \rightarrow$ 1D, 2D or ND signal

$g \rightarrow$ filter/ kernel

with respective dimensions but different size

→ reversed & shifted.

Ex: box blur filter

diff b/w conv. & autocorrelation?

→ in our context, convolutions compose the parameters/weights of a deep neural network.

→ We can build deep neural networks with one/two hundreds of filters to extract hierarchical features from our data.

1D convolution: It is useful for time-series ex: inertial sensors and measuring devices. It is a filtering operation.

For discrete signals we replace integral ($\int$) with summation ($\sum$).

→ Filter g is convolved with signal f by sliding kernel over signal f .

Ex: $f = [5 \ 1 \ 3 \ -1 \ 6 \ 4 \ -4 \ 2 \ -3 \ 1]$

$g = [\frac{1}{3} \ \frac{1}{3} \ \frac{1}{3}]$

$f * g = [- \ 3 \ 1 \ \frac{8}{3} \ 3 \ 2 \ \frac{2}{3} \ -\frac{5}{3} \ 0 \ -]$

We need to decide how to deal with missing values at the boundaries.

same prob. with 2D, 3D or ND convolutions.

2 approaches

1) Drop the dimensions, i.e. convolved sig. is shorter/smaller than original signal.

2) Pad the missing values

- Pad with a constant. Ex: in images pad with zeros

- Mirror from available convolved values.

- replace missing values with mean of available convolved values (can use size of kernel as ref. for computing mean).

→ in sig. processing filter values are fixed, but in deep networks filter values are learned.

→ stride is hyper-parameter & so is kernel/filter size.

→ padding helps to retain sig. dim. after conv.

2D conv.

Image 5x5

-5	3	2	-5	3
4	3	2	1	-3
1	0	3	3	5
-2	0	1	4	4
5	6	7	9	-1

kernel 3x3

0	-1	0
-1	5	-1
0	-1	0

Output 3x3

$$= \begin{bmatrix} 6 & 1 & -8 \\ -7 & 9 & 2 \\ -5 & -9 & 3 \end{bmatrix}$$

→ shift kernel in both dim. & compute resp. by element-wise multiplication

→ stride & kernel size = hyper-parameters

→ one of padding strategies apply here as well.

operation applies to 3D/4D/ND sig. as well.

in addition we can make use of multiple filters different operations.

How many row/col. to pad to get o/p as same size as i/p

What is limitation of high dim. convolutions?

Where each filter expresses a

- filters/kernels applied on data to extract features.
- we also need to learn the values of the filters apart from applying them to extract features.

CNN:

A convNet is a composition of conv. operations of multiple filters, combined with activation functions & other differentiable operations. It approximates a function to map up data to some output.

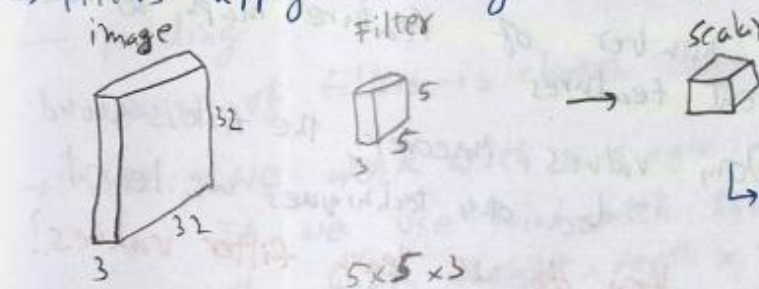
- conv. layer is usually composed of multiple filters/channels, filters also called channels.
- popular for images but also employed for sequential & high dim. problems.

What is req. of data structure for use in conv. layers?

Ex of Conv on RGB image of size 32×32

- 1st filter has same depth as input signal, i.e. 3 for RGB image.

- filters apply spatially but they are 3D.



Filter is convolved with image i.e. dot product at each possible location.

→ result of applying filter at a location is a scalar (z_i).

It is dot product b/w filter parameters/weights W and image patch x_i . In addition, we have a bias term.

- For 1 conv., n.o. of parameters are

$$5 \times 5 \times 3 + 1 = 76 \text{ including bias term}$$

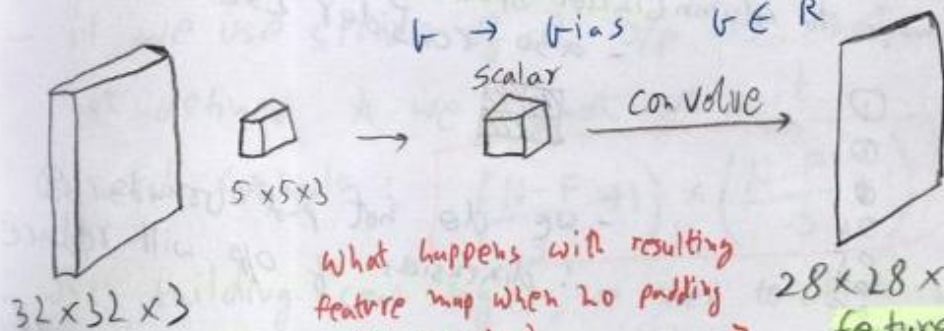
- number of params also represent n.o. of dimensions

- One conv. operation is written as $z_i = W^T x_i + b$

$x_i \rightarrow$ image patch $x_i \in \mathbb{R}^{5 \times 5 \times 3}$

$W^T \rightarrow$ kernel $W^T \in \mathbb{R}^{5 \times 5 \times 3}$

$b \rightarrow$ bias $b \in \mathbb{R}^1$

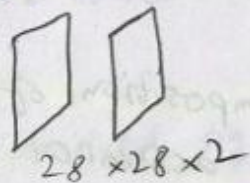


What happens with resulting feature map when no padding is used?

$28 \times 28 \times 1$ feature map

→ feature map is obtained by sliding kernel in both spatial dim. for computing values of z .

→ we can introduce a 2nd filter with different weights/parameters & apply it same way to generate a 2nd feature map with different type of features generated.



→ if we apply 5 diff. kernels it results in 5 diff. feature maps/activation

Why is it important to use diff filters/kernels?

→ to capture different features.

Convolution Layer

- preserves spatial structure
- filters are 3d for images
- conv. layer is differentiable

What happens in fwd & bckwd pass of a conv. layer?

MLP vs Conv. Net

→ diff. is in connectivity, one is dense & other isn't dense.

→ number of filters corresponds to number of feature maps to be learned. Each filter captures different features

→ filters are initialised with random values. Moreover, the filters should not be the same.
 but other techniques we learnt.

How do we learn filter values?

Back prop.

- works in same way as with FC layer.
- only diff. is in connectivity b/w 2 layers
- learning is still accomplished with gradient descent.

$$W = W - \eta \nabla_W L(f(x; W), y)$$

Example with a small gray-scale image:

- consider image/input to be a 3×3 matrix
- 1st reshape it with from image to column (im2col operation)
- also consider 2×2 kernel

1	2	3
4	5	6
7	8	9



①

②

③

04

05

06

07

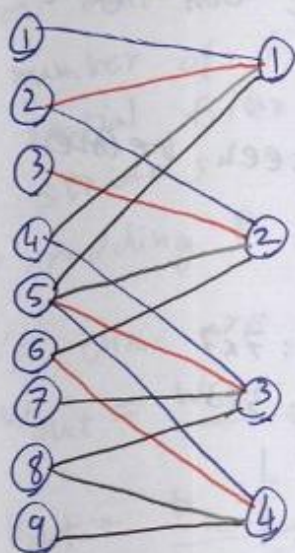
08

09



- we do not pad boundaries
∴ dimensions of o/p will reduce

- kernel is represented by colored connections
- resulting 2×2 table is also represented as columns



→ connectivity same for FWD & backward step.

→ can use standard backprop & gradient descent

→ bias will be another node in 1st column, if we use it.

$$\left(1 + \frac{1+1+1+1}{2}\right) \times \left(1 + \frac{1+1+1+1}{2}\right)$$

Dimensions computations
output dimensions are affected by the

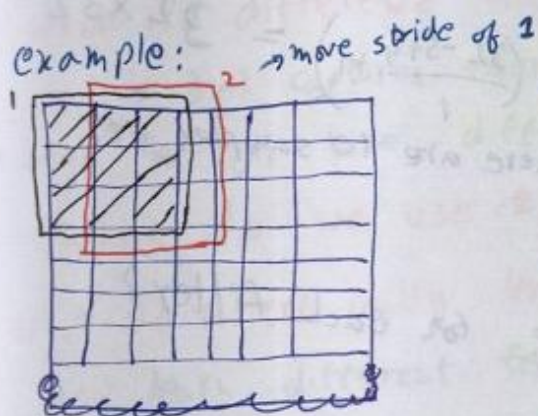
- filter size
- stride
- padding

number of filters is chosen based on experience.

→ note: we work with a single image.

If we use mini-batch then we work with 4th dimensional tensor of width $\times$ height $\times$ depth $\times$ number of samples or filters

example:



- consider a 7×7 image, 3×3 filter with stride 1 & no padding

then o/p is 5×5

- since we don't pad we loop: 1 row from top & bottom; 1 col from left & right.

- if only stride is made 2, output is 3×3

- if we use stride of 3, o/p is not determinable as it is not defined & we cannot use it.

General formula:

$$\left(\frac{N-F}{S} + 1\right) \times \left(\frac{N-F}{S} + 1\right)$$

Input: $N \times N$
Filter: $F \times F$
stride: S

- when building conv. layer, we need to check network def. whether hyper-parameters are compatible.

Ex:

- after 3 conv. we lose $\frac{1}{3}$ of i/p dimensions.
- since we aim to learn features in spatial domain, we would like to retain spatial dimensions. Hence we can make use of padding.
- can make use of any of 3 padding strategies, seen before.

General formula when using padding:

$$\left(\frac{N - F + 2P}{s} + 1 \right) \times \left(\frac{N - F + 2P}{s} + 1 \right)$$

Ex:

Input: 7×7

filter: 3×3

padding: 1

stride: 1

O/p: 7×7

How to pad in order to retain data dim after 3×3 conv?

Given filter $F \times F$, padding for each dim. is $P = \frac{(F-1)}{2}$.

Q. Input image: $32 \times 32 \times 3$ $\rightarrow$ depth for RGB image

10 filters of: 5×5

stride: 1

padding: 2

What is dimension of o/p? $32 \times 32 \times 10$

$$\left(\frac{N - F + 2P}{s} + 1 \right) \times \left(\frac{N - F + 2P}{s} + 1 \right) \\ = \left(\frac{32 - 5 + 4}{1} + 1 \right) \times \left(\frac{32 - 5 + 4}{1} + 1 \right) = 32 \times 32$$

What is size of o/p tensor if there are 10 samples per mini-batch?

$$32 \times 32 \times 10 \times 10$$

How many parameters do we have for each filter?

$$5 \times 5 \times 3 + 1 = 76 \text{ including bias}$$

Thus we have $10 \times 76 = 760$ parameters in total

summary:

i/p to conv. layer is $W_{in} \times H_{in} \times D_{in}$

For conv. we need to choose

- number of filters k
- spatial filter dim. F
- stride s
- padding p

O/p dim. are $W_{out} \times H_{out} \times D_{out}$

$$W_{out} = \frac{W_{in} - F + 2P}{s} + 1$$

$$H_{out} = \frac{H_{in} - F + 2P}{s} + 1$$

$$D_{out} = k$$

We usually use

- std. filters $(F, s, p) = (3, 1, 1)$
- large filters $(F, s, p) = (5, 1, 2)$
- dimensionality reduction $s=2$
- depthwise conv $F=1 \quad s=1 \quad p=0$

parameters of a filter = $F \cdot F \cdot D_{in}$
total params. per conv. = $(F \cdot F \cdot D_{in}) \cdot k$
weights & k bias term

Where do we use depth-wise conv?
→ to compress number of dimensions.

Note: filter can be rectangular, hence dimensions of $m \times n$

What is difference b/w 3×3 & 5×5 kernel?

→ 3×3 captures finer details / information is more local.

→ large kernels have difficulties in optimisation.

When do we use a 5×5 or 7×7 kernel?

What do filters really learn?

They learn different features of the data, like

- edges
- corners
- blobs
- bars
- curves

higher level features include

- skeleton
- eye

How can we reduce size of i/p tensor using conv. kernel?

convolution observations

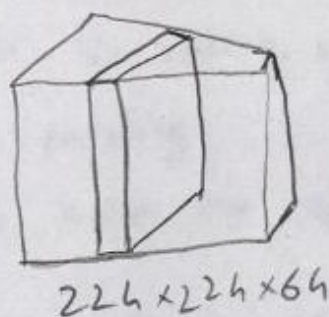
- is used to learn features by applying kernels
- can resize/reshape i/p tensor
- always combined with activation function
- batch normalisation can be added.

→ due to compute cost, i/p is gradually downsampled.

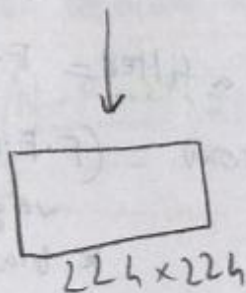
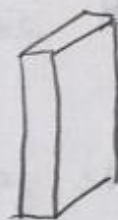
→ alternative to conv. for i/p size = pooling.

Pooling operation

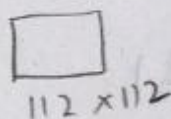
→ reduces dimensions of tensors.



Pool →



down sampling →



Max Pooling:

One channel of i/p tensor

3	1	3	5
6	0	7	9
3	2	1	4
0	2	4	3

Max Pool
2x2 Filter
Stride=2

Max Pooled o/p
6 9
3 4

same param →

Avg Pooled o/p
2.5 6
1.75 3

Which of the 2 pooling operation mimics the brain?

→ Pooling operations are non-differentiable

→ average pooling exists as well.

→ in Max pooling gradient flows only from maximum cell.

→ in Avg pooling gradient flows back from all i/p cells.

→ it doesn't contain learnable parameters.

Dimensions:

i/p tensor: $W_{in} \times H_{in} \times D_{in}$

→ same hyper parameters like in conv:

spatial filter: F

stride: S

Padding: NA as we want to reduce dimensions.

o/p tensor: $W_{out} \times H_{out} \times D_{out}$

common params: $(F, S) = (2, 2)$ or $(3, 2)$

$$W_{out} = \frac{W_{in} - F}{S} + 1$$

$$D_{out} = D_{in}$$

$$H_{out} = \frac{H_{in} - F}{S} + 1$$

Fully Connected layer [last layer in convNet if we want classification]

- convNet has 1 or more FC layers at the end.
- classic FC layer, i.e. all to all connection layer is used.
- last FC layer is usually the trainable classifier/regressor
- once data reaches FC layer, notion of spatial operation goes away. FC layer connects all ip neurons with all neurons of following layer.
- FC layer is followed by activations & combined with dropout if necessary.

How to design ConvNets?

Factor 1: capture different types of features [low, high & middle]

- Factor 2: increase effective receptive field of the network.
- receptive field, i.e. filter size F represents spatial extent of neuron connectivity
 - effective receptive field of neuron represents accumulated spatial extent of connectivity from 1st layer until current layer. It tells us the n.o. of pixels a neuron can see from ip image.
 - applying 3×3 conv. results in a new value that has seen 9 ip pixels/values.
 - goal is to increase effective receptive field of o/p. If possible we would like each neuron to see whole ip image before FC layers. But is expensive, hence we choose pooling as a greedy sub. to keep no. of parameters & dimensions low.

Factor 3: Design of h/w is data dependent

- h/w which applies to image not compatible with audio.
- temporal data might work only with 1D convolutions.
- Network Architecture Search (NAS) & automatic machine learning, actively growing to try to automate design of network.

Factor 4: Human Experience

Deep neural nets. require practical experience for designing well suited-architecture

What are low, high & mid level features in images?

Ch-7 Modern Deep Architectures

1) LeNet (Image data)

- classification of 10 digits [MNIST data]

- composed of
 - conv. layers
 - Tanh activations
 - subsampling
 - FC layers.

- has 5 diff. versions. has around 60k parameters.

- still used for small scale classification tasks

(32x32x1) gray-scale dataset
What is shape of 1st filter?

How many filters does 1st layer have?

As no becomes deeper,
no. of filters increase.
Why not have many filters
in 1st conv. layer?

conv → tanh → subsampling → conv → tanh → subsampling →

FC → Tanh → o/p.

- conv. has kernel of size 5x5 with stride 1

- 2x2 average pooling = subsampling layer

- training not all features propagated to save computations & learn diverse patterns

• o/p layer has 10 euclidean radial basis func. neurons

• L2 dist. is used

What is diff. b/w LeNet's o/p & loss func. compared to modern classification formulations?

O/p func: Tanh/sigmoid

V/s

SoftMax

loss func:

MSE
(not optimal for classification)

V/s

Cross Entropy
(suited for classification as it improves gradient behaviour & avoids vanishing gradient)

↳ MSE can cause in classification

2) AlexNet (Image data)

ILSVRC [ImageNet Large Scale Visual Recognition Challenge]

(60M parameters)

- i/p image: 227x227x3

- o/p prob. prediction for 1000 classes

- ReLU activation, final activation → SoftMax.

- dropout at FC layers

- Local resp. normalization used after activation. predecessor of batch Normalisation that aimed to normalize "feature brightness". (Not used anymore)

- Trained with SGD momentum.

What loss func. is used?

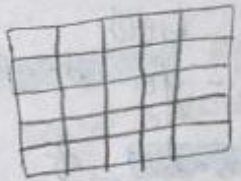
- cross entropy

3) VGGNet (Image Data)

- goes deeper than AlexNet, with few simplifications on n/w design
- drop large kernels in conv. layers. ~~5x5 conv.~~
- 5×5 conv. represented by 2 consecutive 3×3 conv.
- Non-linearity added b/w conv.

Receptive field remains same but parameters drop with 2 consecutive conv.

- $5 \times 5 = 25$ v/s $3 \times 3 + 3 \times 3 = 18$
- stride 1 & padding 1 to retain spatial dimensions
- Max pooling with 2×2 kernel & stride 2
- Local resp. Normalization is Removed.



Rule: Build neural nets with kernels as small as possible. Add as many non-linearities as possible.

Training setup

- Mini-batch size: 256
- LR 10^{-2} , drop by 10 when val. error stabilizes (3 times)
- weight decay: 5×10^{-4}
- SGD with momentum
- Dropout at the FC layers

How to replace 7×7 kernels to reduce n.o. of params?
 → Use 3, 3×3 kernels which also maintains receptive field. $\times 2$

* New receptive field = Prev. receptive field + (K-1) stride

K → kernel size (3×3 in this case)

stride = 1 [assuming no downsampling]

padding = 1 [maintain same spatial dim.]

1st 3×3 conv: receptive field = 3×3

2nd 3×3 conv: receptive field = 5×5 [$3 + (3-1) \times 1$]

3rd 3×3 conv: receptive field = 7×7 [$5 + (3-1) \times 1$]

4) GoogleNet (Image Data)

Motivation: To extract consistent object features under variations, we need to

- employ filters with diff. receptive fields/size.

large filters $\rightarrow$ capture global info.

small filters $\rightarrow$ " local context

- stack multiple filters next to each other to increase effective receptive field.

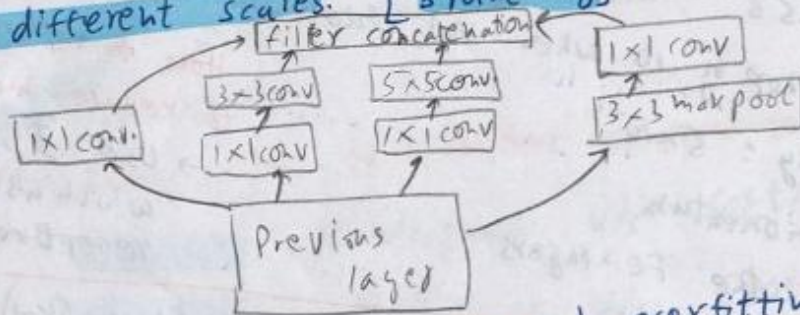
- go deep in w/w architecture

disadv. of above steps:

- large amount of sequential conv. layers increases computational load.
- increasing no. of params can lead to overfitting.
- very deep w/ws face vanishing grad. problems.

Inception Module

- Branch i/p to multiple filters of diff. size and capture features at different scales. [Branching]



This solves prob. of sequential layers & potential overfitting, but still have prob. of vanishing gradients.

- To avoid prob. of vanishing grad., intermediate auxiliary loss functions, i.e. classifiers are used. Intermediate softmax nodes produce additional gradients to avoid vanishing gradients.

Architecture: 9 inception modules, 22 (27 including max pool) layers, 5M params. Global avg. pooling at end of last inception layer. Takes tensors & pools them in a vector. FC & softmax layers follow.

Inception v2/v3: Expand filter banks in inception module. [factorize 5x5 conv into 2 3x3 conv, nxn conv. into 2 1xn & nx1 conv.] Use RMSProp optimization, batch normalization & label smoothing to prevent overfitting.

Inception v4: Reduction blocks to reduce width & height introduced. Inception-ResNet presented.

5) ResNet (Image Data)

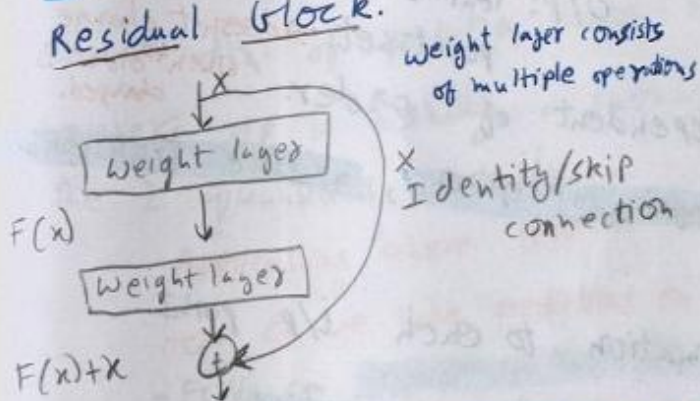
Problem: Deep Neural Networks stop improving at some point when adding conv. layers.

→ why? → weak signals during fwd pass & vanishing gradients cause performance saturation.

Sol: Deep residual learning.

Motivation: Within a neural network, we usually learn a function $H(x) = F(x)$. $F(\cdot)$ corresponds to 1 or more layers of the network. Instead of learning function in one step, we can learn it incrementally as a sum of offsets, i.e. $H(x) = F(x) + x$.

Residual Block:



→ stack multiple residual blocks together to build a deep neural network.

→ ResNet can be represented as a plain n/w too, but cannot become deeper due to a good performance, the connections are necessary.

— during summation if dimensions of x & $F(x)$ don't match, there is a projection conv. at x to make sig. compatible.

Training:

- He parameter init
- SGD with momentum
- LR 0.1 → divide by 10 after perf. saturation

- weight decay 10^{-5}
- minibatch size 256

ResNeXt: idea → build ResNet bottlenecks not only deep but also wide.

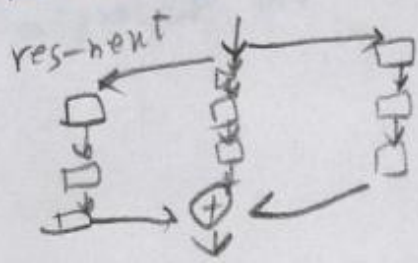
DenseNet: idea → one skip connection is not enough.

— explore how to add multiple skip connections to propagate more info from earlier layers.

dense block is represented as $y = h(x_1, x_{1-1}, \dots, x_{1-n})$

lessons from ResNet:

- skip connections useful for propagating information from earlier layers to later layers.
- deep & wide neural nets learn more meaningful feature representations than sequential deep networks.
- Batch normalization is essential for n/w convergence.



6) Point Net (3D point cloud)

- 3D point cloud is a set of points where each point is described by x, y & z euclidean coordinates
 - Unlike rendered image or 3D voxel rep., it is represented by a list.
 - point cloud is close to raw sensor data.
 - deep learning on a list of points is not possible with conv. nets due to lack of structure in data.
- i/p: unordered point set
- [Global feature]
o/p: feature vector that describes point set. → doesn't change when order is changed.
- o/p representation remains same, independent of order.
 - PointNet is based on multi-layer perceptrons, affine transformations and max pooling operations.
 - T-net applies an affine transformation to each i/p point.
 - Used for point cloud classification & object segmentation
 - Applicable for any kind of unordered list of points.
 - Points from depth cameras, LiDAR or radar sensors.
 - Which is more expensive? PointNet with MLP or w/ 3D conv. or voxels?
 - VoxelNet: transforms point cloud into 3D voxels. Point cloud is thus encoded as volumetric representations.
 - octNet: uses voxel rep. as well, but at diff. resolution. Can work with 3D sparse data.
 - What are adv. & disadv. of voxel based methods?

7) Fully Conv. Networks (structured data)

- combination of FC layers at end & convolutions.

i/p: converted into tensor & later into a vector to make final prediction.

Is it possible to classify each pixel?

→ Fully conv. network represents an architecture where o/p has spatial dimensions.

- idea is to o/p a tensor representing the multiple predictions.

dim. of prediction tensor need not be equal to i/p dim.

- however o/p is normally a submultiple of i/p dim. in order to relate the 2 quantities easily.

Applications where FCN is necessary?

How can we make predictions on spatial res. of i/p?

- FCNs can recover spatial dim. with transposed convolutions (de-conv.) or un-pooling (inverse pooling)

- common to use skip connections before/after dim. recovery. Features from earlier layers help to recover dimensions & learn features.

- FCNs popular for image segmentation & compose part of region-based object detectors.

- Training: Back propagation & stochastic gradient descent.

- Obj detector ~~is~~ faster-RCNN is also based on FCN. SSD / YOLO also purely built on FCN architectures.

8) WaveNet (Audio Data)

Processing sequential data is expensive. size of data/waveform depends on sampling rate & bit depth.

Ex: sampling freq. $44.1 \text{ kHz} = 44100 \text{ elements/second}$.
16 bit sound = 65536 depth ^{or} values

Probr is challenging if we need 1D conv. to process this vast data.

- to process this data Wave net uses

✱ (PTO)

1) Dilated Conv.
 1×3 $[0.3 \ 0.2 \ 0.4]$ can be rep. as 1×5 $[0.3 \ 0 \ 0.2 \ 0 \ 0.4]$
What is adv. of transforming from 1×3 to 1×5 ? $0 \rightarrow$ holes

2) Dilated causal conv.
Conv. kernel here depends only on past values unlike
std. conv. where kernel is centered at current value &
makes use of elements from past & future.

3) Residual Connections
- help build very deep neural networks.

- dilation of 1 = normal conv.

- causal conv. makes use of only past elements.

Applications of wave-net:
- speech generation
- text to speech
- wavenet is generative model.

Efficient inference.
Deep architectures have large memory footprint, energy requirement to
perform inference.

Devices with ltd resources remains a challenge

- sol: 1) build powerful embedded device
2) " learning ops on hardware
3) " small neural nets that behave as good as large ones
↳ refers to neural net. compression.

How can we make neural net. smaller & faster?

- Parameter binarization / quantisation.
- Parameter pruning: prune individual parameters or complete filters of conv. Nets.

Ch-8

Autoencoders

Motivation: to reduce dimensions of high-dimensional data. AE is 1 way to achieve this.

Why reduce dim. of data? → to visualise data → reduce redundancies

How to reduce dim.? → linear approach (PCA) → non-linear (AE)

High dimensional data interpretation not easy.

- common assumption that data lies on low-dimensional non-linear manifold, within higher dim. space.

Goal: build algo that maps high-dim. to low dim. non-linear manifold. i.e. it performs non-linear dimensionality reduction (NLDR).

Ex: Reducing grayscale images to 2D space, ex. to 2 values relation

A scale.

Adv: reducing dim. to 1D, 2D, 3D can make easy visualizations.

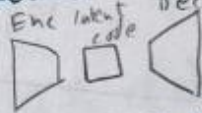
AE is a neural net. that is trained to reconstruct its i/p. Composed of encoder, latent code & the decoder.

Encoder → $f(\cdot)$ transforms i/p x to latent code h $[h = f(x)]$

Decoder → $g(\cdot)$ reconstructs i/p x from latent code h $[\hat{x} = g(h)]$

- Overall, AE tries to copy i/p.
- Since latent code has usually smaller dim. than i/p, it can capture useful info in latent code to reconstruct i/p data.

Illustration of AE:



What is suitable loss for AE? → can use MSE

What type of layers required in decoder to reconstruct i/p? → skip connections → upsampling [extrapolation]

- latent code is a vector/tensor of much lower dimensions than i/p image.
- AE popular for images, but i/p can be of any dimensions & type.

i/p: x o/p: $\hat{x}$

h → low dimensional rep. of i/p

Goal: $x \approx \hat{x}$

For set of training samples loss $L = \|x - \hat{x}\|^2$

where $\hat{x} = g(f(x))$

- Learning to copy i/p is known as recirculation

Advantage of AE v/s PCA?

- AE learns in an unsupervised manner
- AE closely related to generative models.
- latent code smaller in dim. than i/p then AE is called undercomplete.
vice-versa is over-complete.
- Learning a linear decoder is similar to spanning the subspace, similar to PCA.
- Nonlinear decoder/encoder with right capacity can be more powerful than PCA.
- Problem with AE: over fitting
increasing capacity can result in memorizing data & not learning a useful latent code.
- Regularization approaches proposed to avoid overfitting & constrain latent code to learn useful information.

Regularized AE (Under-complete)

idea: to impose constraints during learning for extraction of useful info from input to the latent code.

constraints can be defined as:

- weight decay
- sparsity of latent code $\rightarrow$ sparse AE
- noise on i/p to become more robust
 $\rightarrow$ Denoising AE

- small gradients during param update
 $\rightarrow$ contractive AE

- For each of these models, we consider MLP as neural network
- latent code is represented as a vector.

Weight Decay

std. L2 regularization that we include in training deep neural network for supervised learning.

$$L_{RAE} = L + \beta ||w||^2$$

; $\beta \rightarrow$ controls influence of regularizer.

Sparse AE

(used in past, not anymore)

Additional sparsity constrain on loss func. $L = \|x - \hat{x}\|^2$.
idea: constrain n.o. of activated hidden units at the same time.

- common activation functions usually range from 0 to 1. each value is close to 1 when unit is activated & close to 0 when deactivated.
- sparsity constraint means that our aim is that many activations are near zero. Ex. we can assume to have them at 0.05, thus sparsity parameter will be $r = 0.05$ (hyper-parameter)

consider activation function a_j^l of the layer l & a neuron j
passing neurons from activation, we get:

$$\hat{r}_j^l = \frac{1}{n} \sum_{i=1}^n a_j^l(x^i)$$

objective is to achieve $\hat{r}_j^l \approx r$

- introduce constraint in loss function as a penalty term.
- use KL divergence to measure diff. b/w 2 Bernoulli distributions.

1st one has mean $\hat{r}_j^l$ & 2nd one r

$$L_{SAE} = L + \beta \sum_{l=1}^L \sum_{j=1}^{L_{neur}} D_{KL}(r \| \hat{r}_j^l)$$

$$= L + \beta \sum_{l=1}^L \sum_{j=1}^{L_{neur}} r \log \frac{r}{\hat{r}_j^l} + (1-r) \log \frac{1-r}{1-\hat{r}_j^l}$$

$L \rightarrow$ n.o. of hidden layers
 $L_{neur} \rightarrow$ n.o. of neurons
 $\beta \rightarrow$ hyper-param controls influence of penalty term

Denoising AE (K-sparse AE)

- consider a case of a shallow AE.

How can we choose how many activations to completely cancel?

idea of K-sparse AE: constrain n.o. of deactivated activations.

rule is to keep only K-highest activations.

Algo: 1) Per each feed fwd step & compute all activations

- 2) Find K largest activations of the hidden layer & set rest to 0.
- 3) Compute prediction & loss using sparsified activations of hidden layer.
- 4) Back-propagate through K-largest activations
- 5) Iterate until convergence.

To train deeper k-sparse AE

- 1st train a NW with i/p, hidden & o/p layer.
- Once trained add another hidden layer
- train deep k-sparse AE progressively
- can't be trained with many hidden layers at the same time.

Disadv: Training is slow [train 1 hidden layer at a time]

Performance

To evaluate AE

- 1st train
- 2nd trained AE is finetuned on a specific task, ex: classification of digits.

Fine tuning = process of adapting a pre-trained NW on a specific task & dataset

- Observed that when using hidden representation for ~~classical~~ classification task it works slightly better to keep αk activations, where α is another hyperparam. (selected using validation data) & $\alpha \geq 1$

Denoising AE

An AE eventually learns the identity transformation, which can lead to overfitting.

- Denoising AE is approach to avoid over fitting.

Motivation: Humans can recognize an image even if it is corrupted by noise.

i/p: partially corrupted by noise, coming from prior distribution, the stochastic mapping of clean sig. x to a noisy sig. $\hat{x}$ is represented by $\hat{x} \sim q_0(\hat{x}|x)$

q_0 (stochastic mapping) can combine multiple types of prior dist. it can also set number of elements to 0.

O/p: given noisy sig $\hat{x}$ o/p is clean sig. x .

$$L_{\text{DAE}} = \frac{1}{n} \sum_{i=1}^n (x^i - y(f(\hat{x}^i)))^2$$

Minimising this loss forces AE to extract features that contain useful info to reconstruct the clean signal. Ex noise: Gaussian, random, salt & pepper & binary (set elements to 0)

Can we use KL divergence as loss func? $\rightarrow$ Yes Why?

- Denoising AE can be interpreted as manifold learning where low dimensional ~~latent code~~ ^{manifold} is the latent code.

stacked Denoising AE

- extend original shallow denoising AE to multiple hidden layers, closer to deep learning.

Training is performed in an incremental manner.

- 1) train AE with single hidden layer, corrupt i/p with noise
- 2) Add 2nd hidden layer
- 3) Train 2nd hidden layer (only) by corrupting o/p of 1st hidden layer (only). i.e. we pass clean sig. from i/p & corrupt it after passing through 1st hidden layer.
- 4) Add a 3rd hidden layer
- 5) Train 3rd hidden layer (only) by corrupting o/p of 2nd hidden layer.

disadv: slow training process.

Contractive AE

- Another way to add a penalty term next to reconstruction loss

$$L = \frac{1}{n} \sum_{i=1}^n \|x^i - \tilde{x}^i\|^2$$

- penalty term corresponds to Frobenius norm of the Jacobian matrix of the encoder activations w.r.t i/p dimensions.

$$\left\| \nabla_f f(x) \right\|_F^2 = \sum_{i,j} \left(\frac{\partial h_j(u)}{\partial x_i} \right)^2 \quad \begin{matrix} h \rightarrow \text{hidden layer} \\ x \rightarrow \text{i/p to hidden layer} \end{matrix}$$

- Training has similar effect to an of a sparse AE, i.e. penalising Frobenius norm encourages re mapping to the hidden layers to be in neighbourhood of training data. Low values gradients introduce robustness to small i/p variations.

$$L_{CAE} = L + \beta \left\| \nabla_f f(x) \right\|_F^2 \quad \beta \rightarrow \text{tuning const. to regulate penalty effect}$$

- To build deeper representations protocol of denoising stacked AE is follow: Train 1 hidden layer at a time.

How does progressive training differ from supervised learning?

Encoder-Decoder Architectures

- Layerwise training not seen in modern deep architectures.
All layers ~~at~~ trained at same time.
- modern encoder-decoder architectures are often developed for supervised learning now.
- Ex: U-Net A V-Net for medical image segmentation.

U-Net architecture

- reduction of spatial resolution in encoder *
- increase of " " in decoder *
- skip connections to propagate features from encoder to decoder. *
- is it useful to have skip connections from decoder to encoder?
- At the decoder, up-conv. operation is similar to transposed conv. operation.
- feature reduction while approaching o/p layer of decoder. *
- o/p resolution of seg. map is close to i/p image resolution. *

V-net architecture

- * $\rightarrow$ same properties in vnet (common among many encoder-decoder architectures)
- $\rightarrow$ Uses 3D conv. to process volumetric data, in voxel space.
- Prbr: too many computations due to 3D conv.

Unsupervised learning (Enc-Decoder architectures)

- U-Net, V-Net previously discussed have use supervised learning.
- similar architectures possible for unsupervised learning in context of denoising or restoration/recovery.
 - restoration/recovery of corrupted sig. refer to recover the original clean sig.
 - closely related to denoising.
 - common operations to reduce dim? max pooling conv. with step 2
 - Which other architecture uses skip connections? ResNet

advantages of dim. reduction

- deployment to devices with low memory
- use latent code as i/p to neural network instead of original data.
- deployment to devices with limited computational power.
- use of latent code allows to build smaller networks for faster inference.
- easier visualization. Latent code is low-dimensional representation which we can directly visualize or further reduce its dim. with non-learning approaches.

Ch-9

Recurrent Neural Network (RNN)

Until now we saw all Neural Nets that process data from a single time step. The data can also come from a sequence of measurements over time.

Ex: Audio, stock market, text, video, sensor measurements.

RNNs are capable of processing seq. data

Applications: Machine translation, audio generation, text-to-speech

sequence modelling Ex: Natural language Processing (NLP)

consider sentence "Hello deep learning world!"

- we seek to estimate $P(x)$ where x is a word/punctuation.
- we process this sentence by means of joint distr.
- we know each word depends on all prev. words

$$\Rightarrow P(x) = \prod_i P(x_i | x_1, x_2, \dots, x_{i-1})$$

in our example

$x_1 = \text{hello}$

$x_2 = \text{deep}$

$x_3 = \text{learning}$

$x_4 = \text{world}$

$x_5 = !$

joint distr. is given as

$$P(x) = P(x_1) P(x_2 | x_1) P(x_3 | x_1, x_2) P(x_4 | x_1, x_2, x_3) P(x_5 | x_1, x_2, x_3, x_4)$$

$$P(x_1) = P(\text{hello}) \quad P(x_2 | x_1) = P(\text{deep} | \text{hello}) \quad P(x_3 | x_1, x_2) = P(\text{learning} | \text{hello, deep}) \dots$$

we now have 3 options

- 1) seek to determine plausibility of given sentence based on joint distr.
- 2) build RNN to generate sentences given a few words
- 3) build RNN that takes sentences in one language & translates to other.

Challenges:

New problems compared to Feed Forward networks.

- traceability - initialization
- Use RNN when dealing with
 - Large sequences of data with demand on thousands/millions of parameters.
 - RNNs share parameters.
 - Modelling long term dependencies. RNNs have memory to store dependencies.
 - Sequential data of dynamic i/p / o/p. RNNs process arbitrary data sequences.
- we would like to standardize i/p & o/p of the network

Goal: build a neural network that takes words/punctuation marks as i/p

& o/p in the same format.

How do we represent words for compatibility with RNNs?
How do we represent ground truth of multi class recognition problem?

To make sentences to be processed by a neural network, we convert words/punct. into vectors.

- 1) Convert to **1-hot vector**, i.e. a vector of N binary elements.

Everything is zero other than the element that corresponds to the word.

To build vector dimensions, we 1st need to collect all possible elements

- 2) One hot vector goes through a **word embedding module** that converts one-hot vector to a **real-valued vector**.

The word embedding can be a neural network like Word2Vec or use unsupervised learning such as GloVe (Global Vectors for Word Representation)

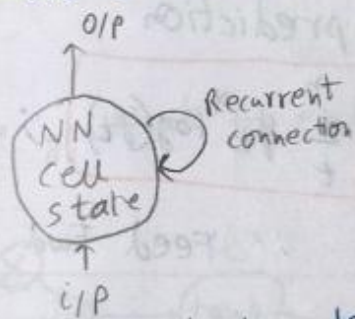
A fastText.

In some cases, like in fastText text can go through w/o 1-hot encoding through the word embeddings.

Word embedding module is NOT trained alongside RNN. It is given and functions as a converter from text to vector.

RNN

- Dimensions of word embeddings are fixed. i/p to RNN has the same dimensions.
- RNN is a neural network with recurrent connections & shared parameters.
- cell state includes the memory of the neural network.



Memory: learns the past dependencies. It is a representation to encode all previously processed information.

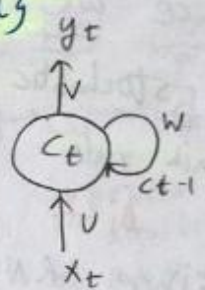
memory representation c_t at time step t captures info from time steps $1 \dots t$. It can be parametrised by a neural network $f(\cdot)$ with parameters Θ .

At $t+1$ memory $c_{t+1} = f_{\Theta}(x_{t+1}, c_t)$

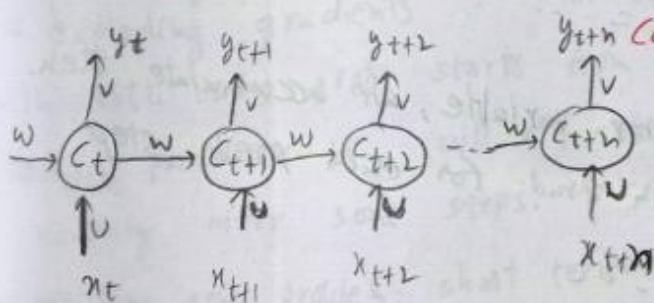
- i/p x at each t has to be transformed to a network representation.
 - internal hidden representation also needs to be transformed to the o/p. this is also task of the neural network $f(\cdot)$ with param. Θ .
- Parameters of these transformations are given by V .

overall we have RNN f with param. $\Theta = \{W, V, U\}$

$W \rightarrow$ represents memory parameters.



- i/p data x_t , processing done with U
- o/p " y_t " " " V
- memory info $\{c_t\}$ " " " W



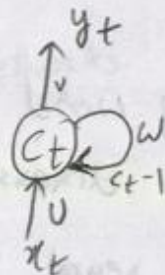
Can we rep. RNN layer wise like convnets?
 → sol: unfold RNN over time to look like feed-fwd network.

if it can be rep. as Feed-Fwd w/o can it be trained like one?

Equations

$$c_t = \tanh(Ux_t + Wc_{t-1})$$

$$y_t = \text{softmax}(Vc_t)$$



- soft-max task ex: choosing a word out of a list within context of translation
- loss is cross entropy for each prediction

$$L = \sum_t^n L_t(y_t, y_t^*) = \sum_t^n y_t^* \log(y_t) ; n \rightarrow \text{n.o. of o/p.}$$

RNN

- have steps instead of classical layers
- share parameters

Feed Fwd n/w

- network layers have individual parameters

RNN Variations:

- n i/p & n o/p
- m i/p & n o/p
- 1 i/p & n o/p
- diff. types of cells, i.e. memory.

Training

- Unfolding RNN brings it to same shape as feed-fwd n/w. Hence we can train RNN with back-propagation over time

A stochastic gradient descent.

Chain rule is used in a similar way to std. Feed-fwd n/w.

Diff. b/w RNN & feed-fwd n/w param. updates?

- since RNN shares parameters there will be multiple gradients for each parameter.
- there will be as many gradients as there are o/p.

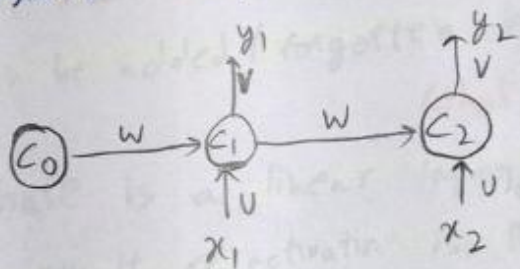
$$\frac{\partial L}{\partial V} = \dots \quad \frac{\partial L}{\partial W} = \dots \quad \frac{\partial L}{\partial U} = \dots$$

Since there are multiple gradients per variable, we accumulate them. for current time step t we obtain grad. for each prev. step

$$\frac{\partial L}{\partial V} = \sum_t \frac{\partial L_t}{\partial V} \dots$$

2-step example

- consider 2 i/p & 2 o/p. $C_0 \rightarrow$ beginning memory [randomly initialised]



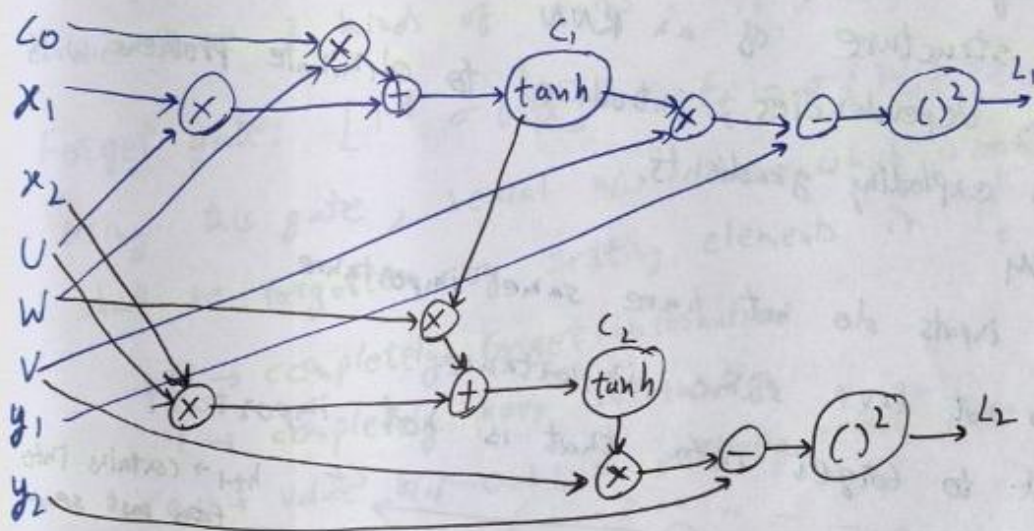
$\rightarrow$ build computational graph similar to feed-fwd n/w using eq.

$$C_t = \tanh(Ux_t + WC_{t-1})$$

$$y_t = \text{softmax}(VC_t)$$

$\rightarrow$ then compute gradients for each variable & sum them up.

- Fwd pass for x_1
- Fwd pass for x_2



$\rightarrow$ compute grad. like in feed-fwd n/w
 $\rightarrow$ update each param with any grad. descent
 $\rightarrow$ Note: sum up multiple grad. for variables U, V, W

- RNNs are flexible structures

Models:

- one-to-one [not RNN, but std. feed Fwd n/w]: image classification

- one-to-many: image captioning

- many-to-one: text to image

- many-to-many: Machine translation / text-to-speech

Gradient problems with RNNs

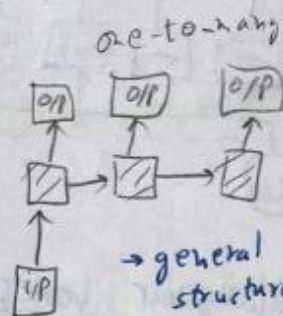
- vanishing gradients
- exploding gradients

$\rightarrow$ unrolling RNN results in deep neural networks & gradient has to travel through

- in both cases prob. starts from modeling long term dependencies. Since we share parameters n/w will be able to be trained even if gradients are vanishing after some steps. What is prob. if we can still update parameters?

- vanishing grad. problem: short term dependencies will be learned & long-term dependencies ignored.

hence unable to learn long-term dependencies.



$\rightarrow$ general structure of RNN with i/p, hidden state & o/p.

- vanishing grad. not easy to solve with engineered solutions.
- exploding grad. can be controlled easily

sol:

1. compute gradient $\frac{\partial L}{\partial w}$
 2. check if below some manually set threshold
 3. if not then re-scale the grad.
- idea similar to grad. clipping. Limitation is we might model only short-term dependencies.

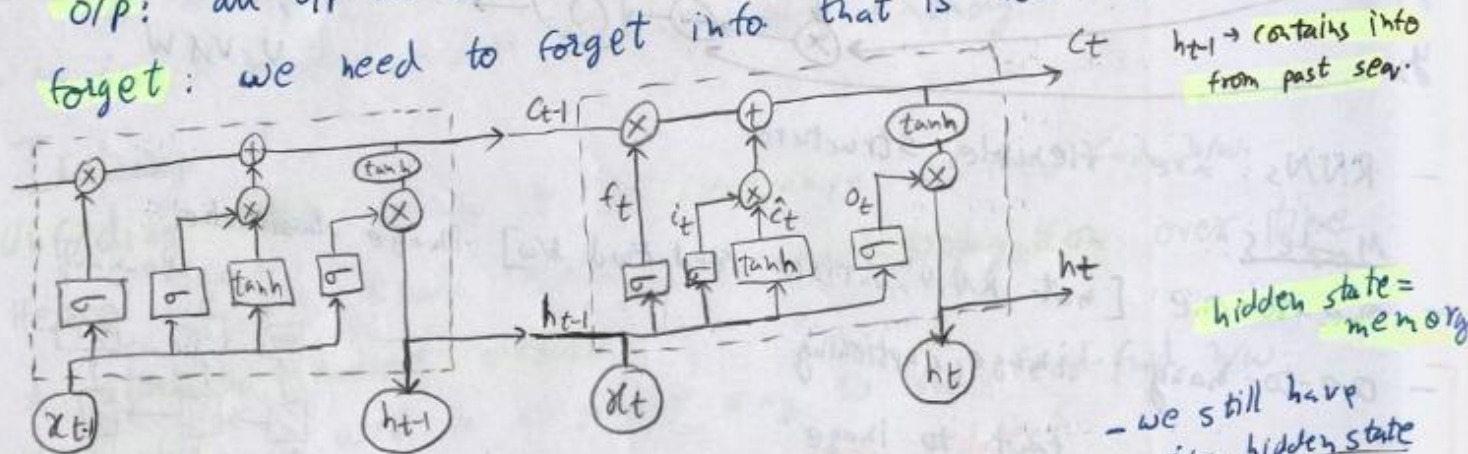
LSTM [Long Short-Term Memory]

- it is a memory structure of an RNN
- LSTM learns long-term dependencies, introduced to eliminate problems with vanishing gradients & exploding gradients.
- Building an LSTM

i/p weights: all inputs do not have same importance

o/p: all o/p do not have same importance.

forget: we need to forget info that is not important



hidden state = memory

- rectangular boxes = neural network layers
- circles = pointwise operations

joint connections of data = feature concatenation

- main diff. to std. RNN is that the cell is more complicated.
- can stack multiple LSTM modules next to each other to build a deep model. [Known approach for seq. modelling with LSTMs]
- Memory & hidden state are diff. entities

- we still have i/p, hidden state & o/p.

- LSTM cells are all operations of the module.

operations:

- cell state C_t can pass through network unmodified, or info can be added / forgotten by 3 gates.

What is LSTM gate?

- Gate is a linear layer with sigmoid activation
- the result of activation is then multiplied ~~with~~ elementwise with data vector.
- this allows select pieces of data to be forgotten / remembered.
- every σ box corresponds to a gate
- we have 3 kinds of gates: i/p, o/p & forget gates. $[h_{t-1}, x_t] \rightarrow \text{concat. of i/p}$

Forget gate: [1st σ box] $f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$

- using this gate, neural n/w learns which information of cell state shall be forgotten by setting elements in f_t to 0/1.

0 $\rightarrow$ completely forget information

1 $\rightarrow$ completely keep information

(0,1) $\rightarrow$ value b/w 0 & 1 = partially retain info

$f_t \rightarrow$ how much of past cell state C_{t-1} should be forgotten.

$W_f \rightarrow$ weight matrix for forget state [learnable parameters]

$b_f \rightarrow$ bias term for forget gate

Input gate: [2nd σ box]

- indicates which part of i/p signal should be stored in the cell.

1. i/p gate decides the values of the cell state which should be updated

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

2. tanh layer results in a vector of values $\hat{C}_t$ that we use to update the cell state.

$$\hat{C}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$$

3. cell state C_t is updated by applying the multiplication of the forget gate & adding the gated new data.

$$C_t = f_t \times C_{t-1} + i_t \times \hat{C}_t$$

Output gate: [3rd σ box]

- decides which parts of cell state are used for new hidden state h_t , based on the prev. hidden state h_{t-1} & input x_t .

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

- tanh activation is applied to cell state C_t before applying it to o/p gate.

this is done to normalise values b/w -1 & 1.

4) o/p of hidden state h_t is computed based on the new cell state & result of o/p gate. It is given by:

$$h_t = o_t \times \tanh(c_t)$$

- hidden state h_t will go to the next LSTM cell or be o/p layer.

Gated Recurrent Unit (GRU)

- is a modification of LSTM network

- main differences are:

- hidden state & cell state are merged
- forget gate & ip gate are combined into single update gate
- ... etc

Ch-10 Deep Generative Models

Generative Model

- models **joint distribution**
 $P(x, y)$
- aim to **model both** hidden & observed variables.
(i/p & o/p)
- Ex: GMM, HMM, Naive Bayes classifier

Discriminative model

- models **conditional distr**
 $P(x|y)$
- aim to **model conditional relation**
b/w some variables (i/p / o/p)
- popular for classification (model a subset of variables)
- Ex: SVM, Random forest, neural networks, CRF

Based on Bayes rule we can switch b/w models

$$P(x|y) = \frac{P(x, y)}{P(y)}$$

Approaches

- Layer-by-Layer: Deep belief Networks, Restricted Boltzmann Machine (RBM)
- Latent variables: Variational Autoencoder
- implicit density: generative stochastic networks (GSN)
- Autoregressive model: Pixel RNN
- Other examples: waveNet, Adversarial learning.

GAN [Generative Adversarial Networks]

- GANs achieve generating images by aligning data & model distribution

idea: align model-generated distribution with real data distr.

- 1) Draw samples (x) from data distribution (P_{data}), i.e. real images.
 - 2) Draw samples from model distribution (P_g) based on set of latent variables (z) (generator w_g)
drawn from prior distr. (P_z), i.e. generated (synthetic) images
 - 3) Detect samples coming from data distr. (real images) with discriminator network. [discriminator w_d] (distinguish P_{data} from P_g)
 - 4) Goal $P_g = P_{data}$. Goal is to achieve Nash-equilibrium
- Nash-equilibrium means neither generator nor discriminator can improve significantly.

- Aim of training algo. is to train both Discriminator (D) & generator (G) at the same time.

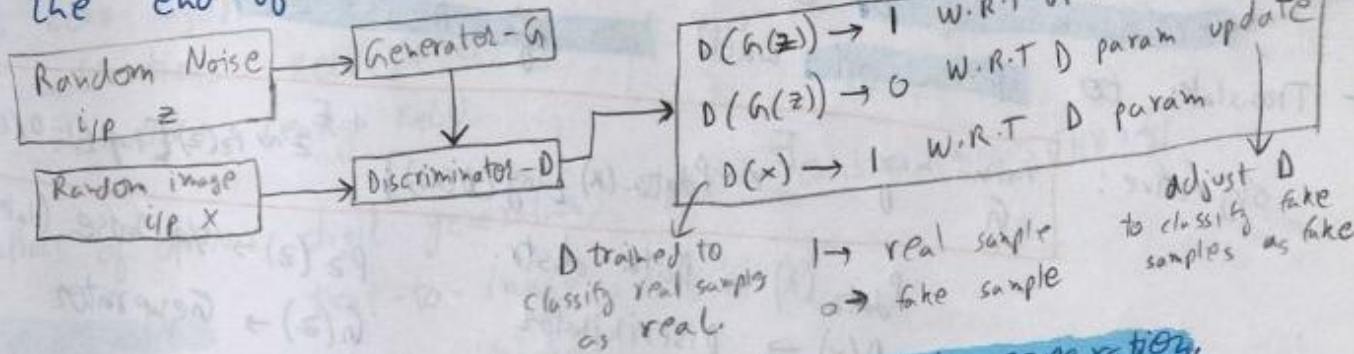
- Idea of having 2 networks compete with each other is called Adversarial training

- model G generates images from input noise

it tries to fool model D

- model D classifies images into real or fake/generated.
it acts as supervision on model G , hence we don't need labels to train the GAN.

- At the end of training we need to keep only the generator (G).



- supervision, provided by D comes from real/fake data separation.

- Training utilises gradient descent & back-prop.

- we need access to real images. Acquiring images is less effort than annotating them

Can GAN like training be used for other data?

What is the challenge in GAN training?

Generator Network

i/p random noise

o/p reconstructed image

objective realistic image reconstruction

loss-2014 No loss

current loss $L1/L2$, perceptual loss

Architecture 2014 Conv, Pool, ReLU

current Architecture Fully conv, U-Net, RNNs

Discriminator Network

generated/real image

class probability

separation b/w generated & real images or model/data distribution

cross-entropy

conv, Pool, ReLU

Limitation of Gen. design?

Training

- K training iterations of
- 1) sample N noise samples z from prior $p_g(z)$
 - 2) sample N samples x from data distr.
 - 3) Update discriminator D (gradient-based)
 - 4) sample N noise samples z from prior $p_g(z)$ [again] [new samples used to update G]
 - 5) Update generator G (gradient based) [typically Generator's loss is based on D 's feedback]

Objective function

- Training GAN is motivated by **minimax game**
- aim: **minimise image reconstruction loss** in generator while **maximising performance of D** to detect real & fake images.
- Translates to **optimal sol: when loss of D is 0.5.**

objective:

$$\min_G \max_D E_{x \sim P_{data}(x)} [\log(D(x))] + E_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

$P_{data}(x) \rightarrow$ Data distr.

$D(x) \rightarrow$ Discriminator

$p_z(z) \rightarrow$ i/p noise (latent variable)

$G(z) \rightarrow$ Generator

Disadvantages

- model collapse for G (**Helvetica scenario**)
generator can learn to produce same image.
His image fills D's expectations. A optimisation will fail, i.e. no more useful gradients produced
- scaling up to deep architectures not possible with MLPs.
orig. GAN could only images smaller than 100×100 pixels.
- G doesn't have an explicit representation. we can't sample from a specific region of distr. we can't marginalize using Re generator

sol. to **Helvetica scenario**: in training do steps 1, 2 x 3 for 5 iterations.

sol. for scaling up to deep architectures: architecture & optimisation changes done

- pooling in G replaced with fractional stride-convolutions (aka deconv.)
- pooling in D replaced with stride-conv.
- Batch normalization (in G & D) introduced
- FC layers replaced with conv. layers.
- last conv. layer in D was reshaped (i.e. Flatten + sigmoid)
- G activations: RELU other than last layer (tanh)
- D activations: Leaky RELU.

Ex: DCGAN
Scaled GAN
for larger
Resolutions

Applications of GAN: digit generation, DCGAN $\rightarrow$ realistic images (HD res)
text-to-image, cycleGAN

Limitations of GANs

- Nash equilibrium cannot be reached
- " " might not exist / degenerate
- Another approach to study GANs is consensus optimisation, by constructing a conservative vector field.

Advantages

- + do not need a sampling method.
using generator, we can infinitely sample images.
- + do not need to model the interferences.
it is a feed-forward step
- + we can approximate a wide range of functions. generating videos, text, audio & other types of data.
- + we do not need labels for training GAN. By relying on real/fake obj objective we can learn to generate images.

Mini-Max Game

Objective: $\min_G \max_D E_{x \sim p_{data}(x)} [\log(D(x))] + E_{z \sim p_z(z)} [\log(1 - D(G(z)))]$

$p_{data}(x) \rightarrow$ data distr. over image x

$D(x)$: Discriminator

$p_z(z) \rightarrow$ data distr. over random noise z

$G(z)$: Generator

$$\min_G \max_D L(D, G) = E_{x \sim p_r(x)} [\log(D(x))] + E_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

$$= E_{x \sim p_r(x)} [\log(D(x))] + E_{z \sim p_g(x)} [\log(1 - D(x))]$$

$p_g(x)$: generator distr. over img x .

$$\Rightarrow L(G, D) = \int (p_r(x) \log(D(x)) + p_g(x) \log(1 - D(x))) dx$$

$L \rightarrow$ global loss with 2 ips Gen & Disc
(G) (D)

$$\Rightarrow L(G, D) = p_r(x) \log(D(x)) + p_g(x) \log(1 - D(x))$$

How to obtain sol. for the loss?

- look 1st for the local minimum of the D

simplify loss assuming:

$$\hat{x} = D(x), A = p_r(x), B = p_g(x)$$

$$\text{loss then becomes: } L = A \log(\hat{x}) + B \log(1 - \hat{x})$$

- now lets find minimum of discriminator D

$$\frac{\delta L}{\delta \hat{x}} = \frac{A}{\hat{x}} - \frac{B}{1 - \hat{x}} \Rightarrow \frac{A}{\hat{x}} - \frac{B}{1 - \hat{x}} = 0 \Rightarrow \frac{A}{\hat{x}} = \frac{B}{1 - \hat{x}} \Rightarrow \hat{x} = \frac{A}{A + B}$$

- re-substituting back our assumptions we get

$$D^*(x) = \frac{p_r(x)}{p_r(x) + p_g(x)} \in [0, 1] \rightarrow \text{bounded b/w 0 & 1 since they are probabilities.}$$

in optimal scenarios, generator will generate a sample that will look

real & D will constantly be fooled.

Expressed as $p_r(x) = p_g(x)$

hence optimal sol. for Discriminator (D) is:

$$D^*(x) = \frac{1}{2}$$

- Now minimising the global loss

since we already have optimum sol. for discriminator & gen.
we substitute below:

$$L(G, D^*) = \int_x \left(p_r(x) \log(D^*(x)) + p_g(x) \log(1 - D^*(x)) \right) dx$$

$$= \log \frac{1}{2} \int_x p_r(x) dx + \log \frac{1}{2} \int_x p_g(x) dx$$

$$= -2 \log 2$$

[$\because$ p_r & p_g are prob. densities the integral over x is equal to 1 and $p_r = p_g$ since they have an optimum generator to create real data]

Distribution Comparison (KL-divergence)

- Divergence measurement of 2 prob. distr. (p) from another expected prob. distr. (q)

- we aim for optimal sol $p(x) = q(x)$

$$D_{KL}(p||q) = \int_x p(x) \log \frac{p(x)}{q(x)} dx$$

- use KL divergence to measure divergence b/w generated samples & real samples. However there is a limitation, KL-div. is asymmetric.

- When $p(x)$ is close to 0 & $q(x)$ significantly non-zero, divergence is zero & the q is ignored.

Jensen-Shannon divergence (JS divergence)

- is a symmetric measure of div.

- compares prob. distr. p & other prob. distr. q

- bounded by $[0, 1]$

$$D_{JS}(p||q) = \frac{1}{2} D_{KL}\left(p||\frac{p+q}{2}\right) + \frac{1}{2} D_{KL}\left(q||\frac{p+q}{2}\right)$$

- aim is to align the 2 distr. p & q

- use JS div. to find when they will be perfectly aligned, i.e. 0 divergence

- For G & D distributions eq. becomes

$$D_{JS}(p_r||p_g) = \frac{1}{2} D_{KL}\left(p_r||\frac{p_r+p_g}{2}\right) + \frac{1}{2} D_{KL}\left(p_g||\frac{p_r+p_g}{2}\right)$$

$$= \frac{1}{2} \left(\log 2 + \int_x p_r(x) \log \frac{p_r(x)}{p_r(x) + p_g(x)} dx \right) + \frac{1}{2} \left(\log 2 + \int_x p_g(x) \log \frac{p_g(x)}{p_r(x) + p_g(x)} dx \right)$$

- we also know $L(G, D^*) = -2 \log 2$; $\rightarrow = \frac{1}{2} (\log 4 + L(G, D^*))$

$$D_{JS}(P_r \parallel P_g) = \frac{1}{2} (\log 4 + L(G, D^*))$$

$$2 D_{JS}(P_r \parallel P_g) - \log 4 = L(G, D^*)$$

$$\Rightarrow L(G, D^*) = 2 D_{JS}(P_r \parallel P_g) - 2 \log 2$$

- Assuming we have optimum generator, divergence measure will be 0.

hence, $L(G, D^*) = -2 \log 2 \rightarrow$ GAN objective is hence equivalent to minimising JS divergence.

